

Neural-network-augmented projection-based model order reduction for mitigating the Kolmogorov barrier to reducibility

Joshua Barnett^{a,*}, Charbel Farhat^{a,b,c}, Yvon Maday^d

^a Department of Mechanical Engineering, Stanford University, Stanford, CA 94305, United States of America

^b Department of Aeronautics and Astronautics, Stanford University, Stanford, CA 94305, United States of America

^c Institute for Computational and Mathematical Engineering, Stanford University, Stanford, CA 94305, United States of America

^d Laboratoire Jacques-Louis Lions (LJLL), Sorbonne Université and Université de Paris Cité, CNRS, F-75005 Paris, France

ARTICLE INFO

Article history:

Received 30 December 2022

Received in revised form 16 July 2023

Accepted 5 August 2023

Available online 10 August 2023

Keywords:

Artificial neural network

Burgers

Kolmogorov n -width

Machine learning

Model reduction

Petrov-Galerkin

ABSTRACT

Inspired by our previous work on a quadratic approximation manifold [1], we propose in this paper a computationally tractable approach for combining a projection-based reduced-order model (PROM) and an artificial neural network (ANN) to mitigate the Kolmogorov barrier to reducibility of parametric and/or highly nonlinear, high-dimensional, physics-based models. The main objective of our PROM-ANN concept is to reduce the dimensionality of the online approximation of the solution beyond what is achievable using affine and quadratic approximation manifolds, while maintaining accuracy. In contrast to previous approaches that exploited one form or another of an ANN, the training of the ANN part of our PROM-ANN does not involve data whose dimension scales with that of the high-dimensional model; and the resulting PROM-ANN can be efficiently hyperreduced using any well-established hyperreduction method. Hence, unlike many other ANN-based model order reduction approaches, the PROM-ANN concept we propose in this paper should be practical for large-scale and industry-relevant computational problems. We demonstrate the computational tractability of its offline stage and the superior wall clock time performance of its online stage for a large-scale, parametric, two-dimensional, model problem that is representative of shock-dominated unsteady flow problems.

© 2023 Elsevier Inc. All rights reserved.

1. Introduction

Projection-based model order reduction (PMOR) allows for the parsimonious representation of a parametric, semi-discrete or discrete high-dimensional model (HDM), through the construction of an associated, lower-dimensional, projection-based reduced-order model (PROM). A second-level approximation known as hyperreduction transforms a PROM into a hyperreduced counterpart (HPROM) capable of operating in real-time, while preserving the desired level of accuracy. As such, a HPROM is essential for many time-critical applications such as simulation-driven design and optimization, optimal control, uncertainty quantification, and digital twinning, to name only a few.

Traditionally, PMOR has been performed using best-approximations on affine subspaces of dimension $n \ll N$, where n and N denote the dimensions of the PROM/HPROM and underlying HDM, respectively. It has demonstrated significant success for a large number of parametric applications grounded in elliptic, parabolic, and second-order hyperbolic partial

* Corresponding author.

E-mail address: jb0@stanford.edu (J. Barnett).

differential equations (PDEs) – for example, problems in solid mechanics [2,3], heat conduction (or diffusion) [4], structural dynamics [5], and wave propagation [6,7]. It has also demonstrated success for parametric, linearized computational fluid dynamic (CFD) and coupled aeroelastic computations [8,9]. For nonparametric RANS [10] and LES [11] CFD computations, PMOR has delivered impressive speedup factors including for compressible flow problems past complex three-dimensional geometries [10]; and in the presence of shocks [12]. For parametric counterpart flow computations – and for that matter, for most first-order hyperbolic PDEs describing transport (or advection) problems – traditional PMOR suffers from the slow decay of the Kolmogorov n -width (for example, see [13]).

The Kolmogorov n -width $d_n(\mathcal{M})$ describes the *worst-case error* with respect to a subset $\mathcal{M}$ of a normed space $\mathcal{S}$ arising from a projection onto the best-possible linear subspace of $\mathcal{S}$ of dimension n [14], represented here by a right reduced-order basis (ROB) $\mathbf{V} \in \mathbb{R}^{N \times n}$. Hence, it sets a limit to any PROM-based approximation; for this reason, it is often referred to as a “barrier” to reducibility. In particular, the Kolmogorov n -width affects most convection-dominated (flow) models; it implies a dimension n that is sufficiently large to challenge the construction of a traditional PROM, or simply defeat the purpose of PMOR.

A variety of approaches has been formulated so far to attempt to mitigate the Kolmogorov barrier to PMOR. Typically, these approaches involve altering the traditional affine subspace approximation to incorporate some nonlinearity. The simplest of them substitutes the affine subspace approximation with a piecewise counterpart [15] constructed by partitioning of the nonlinear high-dimensional solution manifold into regions and computing in each region a local right ROB associated with a local affine subspace approximation. This approach was shown in 2016 to perform in real-time on a laptop, for steady-state RANS computations associated with the NASA Common Research Model [16] and a parametric HDM of dimension greater than $N = 68 \times 10^6$, in the transonic regime characterized by shocks, at a high Reynolds number ($Re = 5 \times 10^6$), and in a shape parameter domain $\mathcal{D}$ of dimension $N_{\mathcal{D}} = 4$ [17]. Nevertheless, due to the Kolmogorov barrier, one can reasonably expect such a PMOR method to face increasing difficulties at delivering a similar performance for increasing values of $N_{\mathcal{D}}$.

Registration methods have also been proposed for addressing the Kolmogorov n -width issue, including in the context of transport problems. From a mathematical viewpoint, these methods transform the traditional affine subspace approximation into a nonlinear approximation by applying a nonlinear parametrization to the affine subspace [18–24] – usually, through a transformation of the given spatial domain. As such, these methods involve the motion and/or deformation of the mesh underlying the semi-discrete or discrete HDM. For this reason, they have been limited so far to simple geometries; or are applicable to complex ones but are in this case vulnerable to mesh entanglements – particularly for large-scale meshes such as those associated, for example, with high Reynolds number flows. From an intuitive viewpoint, registration methods aim at determining domain mappings that cause parametric features of the solution to align in a reference domain for a set of training points sampled in the parameter domain $\mathcal{D}$, which removes the convective nature of the solution, lowers the Kolmogorov n -width, and improves the usefulness of the sampled solution snapshots when an unsampled parameter point is queried in $\mathcal{D}$.

Quadratic approximation manifolds have also been proposed, in two different forms, to mitigate the Kolmogorov barrier: a simulation-free form for structural dynamic applications [25]; and a more general data-driven form that was demonstrated for convection-dominated turbulent flow problems as well as other transport problems [1]. They have also been proposed for performing nonintrusive PMOR using an operator inference method [26].

Most relevant to this paper are approaches that have been proposed for constructing arbitrarily nonlinear approximation manifolds for model order reduction (MOR) using some form of artificial neural network (ANN) representations of the solution or some residual, without necessarily incorporating any projection. One such approach was introduced in [27], in the context of structured meshes, and based on convolutional autoencoders. During training, such methods map the high-dimensional solution to a latent space whose dimension approaches that of the intrinsic solution manifold. Extensions to unstructured meshes were performed in [28], then in [29], using various means, including replacing the convolutional autoencoder with a graph convolutional ANN [29]. The success of all these approaches however comes at a hefty price, as convolutional networks are computationally expensive to train and evaluate, especially for large-dimensional problems whose solutions act as input and output of an autoencoder. Alternative techniques based on conventional deep ANNs (simply referred to in the remainder of this paper as ANNs) have been more recently proposed for mitigating the Kolmogorov barrier. These include the techniques described in [30,31], which use ANNs to learn transformation maps such as those used by registration methods that eliminate the convection nature of the solution, without deforming the underlying mesh however. Another approach that is equally relevant to this paper is the uplifted reduced-order modeling (UROM) method introduced in [32]. This method combines a standard PROM, a memory embedded long short-term memory (LSTM) closure model, and a LSTM-based super-resolution model. Hence, it is a complex approximation method with three modeling layers: a first layer in which a standard, intrusive, PMOR approach is adopted to model the dynamics represented by the largest modes of the system of interest; a closure layer where an LSTM model is introduced to account for the residuals beyond the usual truncation; and a third, uplifting (or super-resolution) layer, which expands the span of the approximation in a post-processing step using a second LSTM.

All ANN-based MOR methods outlined above suffer from either or both of the following issues:

- First, except for the UROM method [32], which qualifies as a PMOR method as it incorporates a projection, the size of their network scales with the large dimension N of the HDM. Hence, for large-scale computational models such as

those associated with flows at high Reynolds numbers and past complex geometries, the training of such networks is computationally intractable. This explains why in most if not all cases, the aforementioned ANN-based MOR methods have been demonstrated on simple one-dimensional applications, occasionally on two-dimensional (2D) ones, in both cases on problems with $N \leq 9000$; and usually without emphasizing offline (training) and/or online (execution) wall clock time. We also note that in the context of *nonintrusive* ANN-based MOR methods however,¹ an approach was most recently presented in [33] for training such methods using projected quantities and therefore a computational complexity that scales with the reduced dimension n of the approximation. This training approach was demonstrated for an application associated with $N = 257\,528$, in which case it required 50 mins on a Tesla V100 32 GB GPU. Methods where ANNs are introduced in the context of Galerkin PMOR to mimic the concept of closure modeling [34,35] also rely on the aforementioned concept of training with projected quantities to achieve computational tractability.

- Second, the hyperreduction of nonlinear reduced-order models (ROMs) constructed using the aforementioned ANN-based MOR methods remains a challenge, although some progress in this area was recently reported in [36] for the case of sparse shallow auto-encoders. As for the case of UROM, which is a PMOR method with two different ANNs, how to incorporate in it a computationally efficient hyperreduction process remains at best unclear.

The present work is inspired by our previous work on mitigating the Kolmogorov barrier, which focused for this purpose on a data-driven approach for constructing a quadratic approximation manifold [1]. It proposes a general, intrusive, *ANN-augmented* PROM that we refer to here and throughout the remainder of this paper as a PROM-ANN. The main objective of the PROM-ANN is, for a fixed level of accuracy, to reduce the dimensionality of the online approximation of the solution beyond what is possible using the quadratic approximation, and dramatically beyond what is achievable using the traditional affine subspace approximation. In contrast to the alternative intrusive and projection-free approaches outlined above for constructing arbitrarily nonlinear approximation manifolds for MOR using some form of ANN, the training of the ANN part of the PROM-ANN we propose in this paper does *not* involve data whose dimension scales with that of the HDM. In contrast to a UROM [32], our PROM-ANN is simpler to construct, relies on a single ANN, and couples it strongly with the affine approximation; therefore, our PROM-ANN is deemed to be more practical and performant than a UROM. And unlike all ROMs resulting from all aforementioned dimensionality reduction approaches, our PROM-ANN is *efficiently* hyperreducible using any well-established hyperreduction method. Consequently, unlike most if not all of the aforementioned ANN-based approaches, the PROM-ANN concept we propose in this paper should be practical for large-scale and industry-relevant problems which, in the context of CFD for example, tend to have hundreds of millions of degrees of freedom (dofs).

While the scope of the PROM-ANN concept we propose here is equally applicable to HDMs arising from the semi-discretization or discretization of elliptic, parabolic, and both types of hyperbolic PDEs, we focus in this paper on the case of first-order hyperbolic PDEs, as the PMOR of the corresponding HDMs is most vulnerable to the Kolmogorov barrier. Consequently, we ground our work in the nonlinear least-squares Petrov-Galerkin (LSPG) PMOR method [37,38] which represents the state of the art of PMOR for convection-dominated turbulent flow problems. However, we note that the approach we propose in Section 3 for constructing an arbitrarily nonlinear approximation manifold using a right ROB and an ANN is equally applicable to Galerkin PMOR methods.

To achieve computational efficiency, we choose to hyperreduce our PROM-ANN using the energy-conserving sampling and weighting (ECSW) [5] method. Although originally designed in the context of finite element (FE) approximations, solid mechanics, and Galerkin PROMs, this hyperreduction method was recently extended in [10] to that of finite volume (FV) approximations, CFD, and LSPG PMOR [10]; and was shown to deliver impressive speedup factors while preserving a desirable level of accuracy. Nevertheless, we note that any other hyperreduction method such as the empirical interpolation method (EIM) [39] and its discrete version (DEIM) [40] can be considered to hyperreduce a PROM-ANN.

The remainder of this paper is organized as follows. In Section 2, we set the scope of this work and overview the LSPG and ECSW methods. In Section 3, we present the main underpinning of the PROM-ANN concept – namely, a computationally tractable approach for constructing an arbitrarily nonlinear approximation manifold using a right ROB and an ANN. In Section 4, we adapt LSPG and ECSW to this concept and comment on the additional computational complexity and additional storage requirement entailed by the adaptations. In Section 5, we apply our proposed PMOR technology to two instances of dimension 125 000 and 1 125 000 of a popular, parametric, shock-dominated benchmark problem that exhibits the Kolmogorov n -width issue. We contrast the performance of our computational technology with that of the traditional PMOR method. We highlight the computational tractability of its offline stage and the superior performance of its online stage. Specifically, for the same level of desirable accuracy, we report impressive wall clock time speedup factors with respect to both the considered HDMs and their traditional HPROMs. In Section 6, we offer concluding remarks.

¹ A nonintrusive ANN-based MOR method is a low-dimensional approximation method where the ANN-based representation is constructed offline from solution snapshots of a discretized PDE of interest, but is not otherwise informed online by the discretized PDE.

2. Background

Throughout this paper, we consider the following parametric, nonlinear, semi-discrete, first-order initial value problem

$$\begin{aligned} \mathbf{M}(\boldsymbol{\mu})\dot{\mathbf{u}}(t; \boldsymbol{\mu}) + \mathbf{f}(\mathbf{u}(t; \boldsymbol{\mu}); \boldsymbol{\mu}) - \mathbf{g}(t; \boldsymbol{\mu}) &= \mathbf{0} \\ \mathbf{u}(0; \boldsymbol{\mu}) &= \mathbf{u}^0(\boldsymbol{\mu}) \end{aligned} \quad (1)$$

where $t \in]0, T_f]$ denotes time and $]0, T_f]$ denotes the time-interval of interest; $\boldsymbol{\mu} \in \mathcal{D} \subset \mathbb{R}^{N_{\mathcal{D}}}$ denotes a vector-valued parameter of dimension $N_{\mathcal{D}}$ and is often referred to as a “point” in the parameter domain $\mathcal{D}$; a dot denotes a time-derivative; $\mathbf{u} \in \mathbb{R}^N$ denotes the semi-discrete solution vector; $\mathbf{M} \in \mathbb{R}^{N \times N}$ is a mass-like matrix; $\mathbf{f} \in \mathbb{R}^N$ is a nonlinear vector function representing a semi-discrete flux or an internal force vector; $\mathbf{g} \in \mathbb{R}^N$ is a nonlinear vector function representing a semi-discrete external force vector, source term, or the effect of some time-dependent boundary conditions; and $\mathbf{u}^0 \in \mathbb{R}^N$ denotes an initial condition for the semi-discrete solution vector. We refer to this problem as the HDM problem (or simply HDM) and consider its solution using an *implicit* time-integration scheme. We refer to the computed solution as the high-dimensional solution (or simply HDM solution). We note that this HDM encompasses counterparts associated with nonlinear, semi-discrete, second-order dynamical systems characterized by arbitrarily complex dissipative (damping) forces, as such problems can be rewritten in the first-order form (1) using a change of variables and some mathematical transformations [5].

2.1. The LSPG method for the dimensionality reduction of a HDM

The traditional affine subspace approximation associated with traditional PMOR can be expressed as

$$\mathbf{u}(t; \boldsymbol{\mu}) \approx \tilde{\mathbf{u}}(t; \boldsymbol{\mu}) = \mathbf{u}_{\text{ref}} + \mathbf{V}\mathbf{q}(t; \boldsymbol{\mu}) \quad (2)$$

where $\mathbf{u}_{\text{ref}}$ is a reference solution; and the right ROB $\mathbf{V}$ is constructed as usual – that is, by adaptively sampling $\mathcal{D} \times]0, T_f]$ using a greedy procedure, solving the HDM problem at the samples $\{(t^i; \boldsymbol{\mu}^j)\}_{i=1; j=1}^{i=N_t; j=N_{\mu}}$, collecting the $N_s = N_t \times N_{\mu}$ computed solution snapshots in the snapshot matrix $\mathbf{S} \in \mathbb{R}^{N \times N_s}$; compressing this matrix using the thin SVD $\mathbf{S} = \mathbf{U}_s \boldsymbol{\Sigma}_s \mathbf{Y}_s^T$, where $\mathbf{U}_s \in \mathbb{R}^{N \times k}$ is the matrix of left singular vectors, $\boldsymbol{\Sigma}_s \in \mathbb{R}^{k \times k}$ is the diagonal matrix of nonzero singular values $\sigma_{s,i}, i \in \{1, \dots, k\}$, stored in the order $\sigma_{s,1} \geq \sigma_{s,2} \geq \dots \sigma_{s,k} > 0$, $\mathbf{Y}_s \in \mathbb{R}^{N_s \times k}$ is the matrix of right singular vectors, $k \leq \min(N, N_s)$ denotes the rank of $\mathbf{S}$, and the superscript T designates the transpose; and identifying $\mathbf{V} \in \mathbb{R}^{N \times n}$ as the first $n \ll N$ columns of $\mathbf{U}_s$, where the dimension n is typically determined such that

$$1 - \frac{\sum_{i=1}^n \sigma_{s,i}^2}{\sum_{j=1}^k \sigma_{s,j}^2} \leq \varepsilon_s^2 \quad (3)$$

where $\varepsilon_s > 0$ is a user-defined tolerance.

Substituting (2) in (1) leads to the nonlinear, semi-discrete, time-dependent residual equation and its associated initial condition

$$\begin{aligned} \mathbf{r}(\tilde{\mathbf{u}}(t; \boldsymbol{\mu}), t; \boldsymbol{\mu}) &= \mathbf{r}(\mathbf{u}_{\text{ref}} + \mathbf{V}\mathbf{q}(t; \boldsymbol{\mu}), t; \boldsymbol{\mu}) = \mathbf{M}(\boldsymbol{\mu})\mathbf{V}\dot{\mathbf{q}}(t; \boldsymbol{\mu}) + \mathbf{f}(\mathbf{u}_{\text{ref}} + \mathbf{V}\mathbf{q}(t; \boldsymbol{\mu}); \boldsymbol{\mu}) - \mathbf{g}(t; \boldsymbol{\mu}) \\ \tilde{\mathbf{u}}^0(\boldsymbol{\mu}) &= \mathbf{u}_{\text{ref}} + \mathbf{V}\mathbf{q}(0; \boldsymbol{\mu}) \approx \mathbf{u}^0(\boldsymbol{\mu}) \end{aligned}$$

The time-discretization of the first of the above equations by a preferred implicit scheme leads at each time-step $m+1$ to a nonlinear system of algebraic equations of the form

$$\mathbf{r}^{m+1}(\boldsymbol{\mu}) = \mathbf{r}(\mathbf{u}_{\text{ref}} + \mathbf{V}\mathbf{q}^{m+1}(\boldsymbol{\mu}), t^{m+1}; \boldsymbol{\mu}) = \mathbf{0} \quad (4)$$

where $\mathbf{r}^{m+1} \in \mathbb{R}^N$ and the superscript $m+1$ (or m) designates here and throughout the remainder of this paper a discrete quantity evaluated at time t^{m+1} (or t^m). The N -dimensional system (4) is overdetermined as it governs $n \ll N$ unknowns – namely, the generalized coordinates of the subspace approximation (2) stored in $\mathbf{q}^{m+1}$. Thus, the unknowns represented by the reduced-order vector $\mathbf{q}^{m+1}$ are constrained by enforcing the orthogonality of the discrete residual $\mathbf{r}^{m+1}$ to a left ROB $\mathbf{W} \in \mathbb{R}^{N \times n}$, which leads to the PROM counterpart of the HDM (1)

$$\mathbf{W}^T \mathbf{r}^{m+1}(\boldsymbol{\mu}) = \mathbf{W}^T \mathbf{r}(\mathbf{u}_{\text{ref}} + \mathbf{V}\mathbf{q}^{m+1}(\boldsymbol{\mu}), t^{m+1}; \boldsymbol{\mu}) = \mathbf{0} \quad (5)$$

In the case of the LSPG PMOR method, the overdetermined system of equations (4) is solved at each time-step for $\mathbf{q}^{m+1}$ using the Gauss-Newton method. In this case, the left ROB is constructed at each Gauss-Newton iteration $\ell+1$ of each time-step t^{m+1} as follows [37,38]

$$\mathbf{W}^{m+1,\ell+1}(\boldsymbol{\mu}) = \mathbf{J}^{m+1,\ell}(\boldsymbol{\mu}) \frac{\partial \tilde{\mathbf{u}}}{\partial \mathbf{q}}(\mathbf{q}^{m+1,\ell}(\boldsymbol{\mu})) = \mathbf{J}^{m+1,\ell}(\boldsymbol{\mu}) \mathbf{V} \quad (6)$$

where the second equality in (6) results from the approximation (2) and

$$\mathbf{J}^{m+1,\ell}(\boldsymbol{\mu}) = \mathbf{J}(\tilde{\mathbf{u}}^{m+1,\ell}(\boldsymbol{\mu}); \boldsymbol{\mu}) = \frac{\partial \mathbf{r}}{\partial \tilde{\mathbf{u}}}(\tilde{\mathbf{u}}^{m+1,\ell}(\boldsymbol{\mu}); \boldsymbol{\mu}) = \frac{\partial \mathbf{r}}{\partial \tilde{\mathbf{u}}}(\mathbf{u}_{\text{ref}} + \mathbf{V}\mathbf{q}^{m+1,\ell}(\boldsymbol{\mu}); \boldsymbol{\mu}), \quad \mathbf{J}^{m+1,\ell}(\boldsymbol{\mu}) \in \mathbb{R}^{N \times N} \quad (7)$$

Remark 1. From (6) and (7), it follows that $\mathbf{W}^{m+1,\ell+1}$ is independent of $\mathbf{q}^{m+1,\ell+1}$ – and therefore independent of $\tilde{\mathbf{u}}^{m+1,\ell+1} = \mathbf{u}_{\text{ref}} + \mathbf{V}\mathbf{q}^{m+1,\ell+1}$; this justifies in (6) the notation $\mathbf{W}^{m+1,\ell+1}(\boldsymbol{\mu})$, which highlights the dependence of $\mathbf{W}^{m+1,\ell+1}$ on $\boldsymbol{\mu}$ only. For the sake of simplicity however, the superscript $\ell+1$ designating the $(\ell+1)$ -th iteration of the Gauss-Newton method is dropped from any double superscript $^{m+1,\ell+1}$ throughout the remainder of this paper, except when otherwise noted.

Now, using the left ROB $\mathbf{W}^{m+1}$ defined in (6) and (7), solving the discrete, nonlinear PROM problem (5) becomes equivalent to solving the optimization problem

$$\mathbf{q}^{m+1}(\boldsymbol{\mu}) = \arg \min_{\mathbf{x} \in \mathbb{R}^n} \|\mathbf{r}(\mathbf{u}_{\text{ref}} + \mathbf{V}\mathbf{x}, t^{m+1}; \boldsymbol{\mu})\|_2^2$$

(see [11] for a mathematical proof).

2.2. The ECSW method for the hyperreduction of a LSPG-based PROM

For many nonlinear problems of practical interest, the complexity of the solution of problem (5) – and for this matter, that of the processing of any nonlinear PROM – scales not only with the small dimension n of the PROM, but also with the large dimension N of the HDM. Hyperreduction [41] is one approach for eliminating this computational bottleneck. In this work, we choose ECSW [5,42] for hyperreducing any constructed nonlinear PROM, because of its track record established in [5,42,10] for many different large-scale applications; and summarize it below.

Let $\mathcal{E} = \{\mathbf{e}_i\}$, $i \in \{1, \dots, N_e\}$ denote the set of $N_e = |\mathcal{E}|$ mesh entities defining the discretization of the computational domain underlying the semi-discrete HDM (1). For example, these entities may be elements in the case of FE modeling, or computational (primal or dual) cells in the case of FV modeling. In all cases, the discrete, nonlinear PROM (5) of dimension n can be rewritten in the case of the LSPG PMOR method as

$$\begin{aligned} \mathbf{r}_n^{m+1}(\boldsymbol{\mu}) &= (\mathbf{W}^{m+1}(\boldsymbol{\mu}))^T \mathbf{r}(\mathbf{u}_{\text{ref}} + \mathbf{V}\mathbf{q}^{m+1}(\boldsymbol{\mu}), t^{m+1}; \boldsymbol{\mu}) \\ &= \sum_{e_i \in \mathcal{E}} (\mathbf{L}_{e_i} \mathbf{W}^{m+1}(\boldsymbol{\mu}))^T \mathbf{r}_{e_i}(\mathbf{L}_{e_i^+} [\mathbf{u}_{\text{ref}} + \mathbf{V}\mathbf{q}^{m+1}(\boldsymbol{\mu})], t^{m+1}; \boldsymbol{\mu}) \end{aligned} \quad (8)$$

where:

- The notation $\dagger[\bullet]$ is used here and throughout the remainder of this paper to indicate that the quantity $\dagger$ is to be multiplied by the quantity $\bullet$ and not that $\dagger$ is a function of $\bullet$.
- $\mathbf{r}_{e_i} \in \mathbb{R}^{d_{e_i}}$ represents the contribution of the individual mesh entity e_i to the discrete residual $\mathbf{r}$ and d_{e_i} denotes the number of dofs attached to e_i .
- $\mathbf{L}_{e_i} \in \{0, 1\}^{d_{e_i} \times N}$ is a Boolean matrix selecting the mesh entity e_i to which d_{e_i} dofs are attached.
- $\mathbf{L}_{e_i^+} \in \{0, 1\}^{d_{e_i^+} \times N}$ is a Boolean matrix selecting the union of the mesh entity e_i and its neighbors participating in the same stencil of the chosen semi-discretization scheme, to which $d_{e_i^+}$ dofs are attached.
- The left ROB $\mathbf{W}^{m+1}$ is given by (6) and (7).

The main idea underlying ECSW is to approximate a projected quantity such as (8) using a cubature approach whose complexity is independent of N , as follows

$$\begin{aligned} \mathbf{r}_n^{m+1}(\boldsymbol{\mu}) &= (\mathbf{W}^{m+1}(\boldsymbol{\mu}))^T \mathbf{r}(\mathbf{u}_{\text{ref}} + \mathbf{V}\mathbf{q}^{m+1}(\boldsymbol{\mu}), t^{m+1}; \boldsymbol{\mu}) \\ &\approx \tilde{\mathbf{r}}_n^{m+1}(\boldsymbol{\mu}) = \sum_{e_i \in \tilde{\mathcal{E}}} \xi_{e_i} (\mathbf{L}_{e_i} \mathbf{W}^{m+1}(\boldsymbol{\mu}))^T \mathbf{r}_{e_i}(\mathbf{L}_{e_i^+} [\mathbf{u}_{\text{ref}} + \mathbf{V}\mathbf{q}^{m+1}(\boldsymbol{\mu})], t^{m+1}; \boldsymbol{\mu}) \end{aligned} \quad (9)$$

where $\tilde{\mathcal{E}}$ is a subset of $\mathcal{E}$ defining a “reduced mesh” – that is, $n_e = |\tilde{\mathcal{E}}| \ll N_e = |\mathcal{E}|$ – and interpretable as the set of points of the cubature; and $\{\xi_{e_1}, \dots, \xi_{e_{n_e}}\}$ is the set of *positive*, real-valued weights of the cubature approximation.

For implicit time-discretizations, it follows from Remark 1 that the Jacobian with respect to $\tilde{\mathbf{u}}^{m+1}$ of a discrete, nonlinear, LSPG-based PROM of dimension n and the form given in (5) can be written as

$$\begin{aligned}\mathbf{J}_n^{m+1}(\boldsymbol{\mu}) &= (\mathbf{W}^{m+1}(\boldsymbol{\mu}))^T \mathbf{J}^{m+1}(\boldsymbol{\mu}) = (\mathbf{W}^{m+1}(\boldsymbol{\mu}))^T \frac{\partial \mathbf{r}(\mathbf{u}_{\text{ref}} + \mathbf{V}\mathbf{q}^{m+1}(\boldsymbol{\mu}), t^{m+1}; \boldsymbol{\mu})}{\partial \tilde{\mathbf{u}}^{m+1}} \\ &= \sum_{e_i \in \mathcal{E}} (\mathbf{L}_{e_i} \mathbf{W}^{m+1}(\boldsymbol{\mu}))^T \mathbf{J}_{e_i}(\mathbf{L}_{e_i} + [\mathbf{u}_{\text{ref}} + \mathbf{V}\mathbf{q}^{m+1}(\boldsymbol{\mu})]; \boldsymbol{\mu})\end{aligned}$$

where $\mathbf{J}_{e_i} \in \mathbb{R}^{d_{e_i} \times d_{e_i}}$ represents the contribution of the individual mesh entity e_i to the Jacobian $\mathbf{J}$ and d_{e_i} denotes as before the number of dofs attached to e_i . Since $(\mathbf{W}^{m+1})^T \partial \mathbf{r}^{m+1} / \partial \tilde{\mathbf{u}}^{m+1} = \partial \left((\mathbf{W}^{m+1})^T \mathbf{r}^{m+1} \right) / \partial \tilde{\mathbf{u}}^{m+1}$, it follows that $\mathbf{J}_n^{m+1}$ is the Jacobian of the projected residual (8). Therefore, $\mathbf{J}_n^{m+1}$ can be approximated using the same cubature rule as in (9), as follows

$$\begin{aligned}\mathbf{J}_n^{m+1}(\boldsymbol{\mu}) &= (\mathbf{W}^{m+1}(\boldsymbol{\mu}))^T \mathbf{J}^{m+1}(\boldsymbol{\mu}) = (\mathbf{W}^{m+1}(\boldsymbol{\mu}))^T \frac{\partial \mathbf{r}(\mathbf{u}_{\text{ref}} + \mathbf{V}\mathbf{q}^{m+1}(\boldsymbol{\mu}), t^{m+1}; \boldsymbol{\mu})}{\partial \tilde{\mathbf{u}}^{m+1}} \\ &\approx \sum_{e_i \in \tilde{\mathcal{E}}} \xi_{e_i} (\mathbf{L}_{e_i} \mathbf{W}^{m+1}(\boldsymbol{\mu}))^T \mathbf{J}_{e_i}(\mathbf{L}_{e_i} + [\mathbf{u}_{\text{ref}} + \mathbf{V}\mathbf{q}^{m+1}(\boldsymbol{\mu})]; \boldsymbol{\mu})\end{aligned}\quad (10)$$

Specifically, ECSW computes the reduced mesh $\tilde{\mathcal{E}}$ and its associated set of weights $\{\xi_{e_1}, \dots, \xi_{e_{n_e}}\}$ using a machine learning approach. Essentially, it trains the approximation (9) – from which the counterpart approximation (10) can be derived – on a subset of the solution snapshots already collected in the snapshot matrix $\mathbf{S}$, as described below.

Let $\mathbf{C} = [c_{le}] \in \mathbb{R}^{N_h \times N_e}$ and $\mathbf{d} = [d_l] \in \mathbb{R}^{N_h}$ be the matrix and vector defined as follows

$$\begin{aligned}c_{le} &= (\mathbf{L}_{e_i} \mathbf{W}^{m+1})^T \mathbf{r}_e \left(\mathbf{L}_{e_i} + [\mathbf{V}\mathbf{V}^T (\mathbf{u}^l - \mathbf{u}_{\text{ref}}) + \mathbf{u}_{\text{ref}}], (t; \mu)^l \right), \quad l = 1, \dots, N_h \\ d_l &= \sum_{e \in \mathcal{E}} c_{le}, \quad l = 1, \dots, N_h\end{aligned}\quad (11)$$

where $(t; \mu)^l$ is a generic notation for a sampled pair (t^i, μ^j) of a time-step and a parameter point, and $N_h \leq N_s$ denotes the size of the subset of the solution snapshots collected in $\mathbf{S}$ chosen for training the cubature approximation (9). Note that in (11) above:

- $\mathbf{V}\mathbf{V}^T (\mathbf{u}^l - \mathbf{u}_{\text{ref}})$ is the orthogonal projection of the collected solution snapshot $\mathbf{u}^l$ onto the subspace represented by $\mathbf{V}$. This emphasizes that the ECSW training is performed for PROM predictions and not HDM predictions, consistently with (9).
- $\mathbf{C}$ and $\mathbf{d}$ verify

$$\mathbf{C}[\boldsymbol{\xi} = \mathbf{1}] = \mathbf{d} \quad (12)$$

where $\boldsymbol{\xi} \in \mathbb{R}^{N_e}$ is the vector of weights of the cubature assuming that $\tilde{\mathcal{E}} = \mathcal{E}$ and $\mathbf{1} \in \mathbb{R}^{N_e}$ is the N_e -long vector with each entry equal to 1. This identity is a rewriting of (8) in matrix form.

Then, ECSW can be simply described as relaxing (12) and defining instead $\tilde{\mathcal{E}}$ and $\boldsymbol{\xi}$ as the solution of the optimization problem

$$\min_{\boldsymbol{\xi} \in \mathbb{R}_{\geq 0}^{N_e}} \|\boldsymbol{\xi}\|_0 \quad \text{s.t.} \quad \|\mathbf{C}\boldsymbol{\xi} - \mathbf{d}\|_2 \leq \varepsilon_{\text{ECSW}} \|\mathbf{d}\|_2 \quad (13)$$

where $\|\bullet\|_0$ designates the ℓ_0 -norm of $\bullet$ – that is, the total number of nonzero entries of the vector $\bullet$ – and $0 < \varepsilon_{\text{ECSW}} < 1$ is a specified training tolerance. Unfortunately, the above minimization problem is combinatorially hard. For this reason, several alternative formulations of (13) were considered in [43] and the following nonnegative least squares approach

$$\boldsymbol{\xi} = \arg \min_{\boldsymbol{\xi} \in \mathbb{R}_{\geq 0}^{N_e}} \|\mathbf{C}\boldsymbol{\xi} - \mathbf{d}\|_2^2 \quad (14)$$

equipped with the early termination criterion

$$\|\mathbf{C}\boldsymbol{\xi} - \mathbf{d}\|_2 \leq \varepsilon_{\text{ECSW}} \|\mathbf{d}\|_2 \quad (15)$$

was found to be most practical and computationally efficient. Due to (15), the solution of the optimization problem (14) is a sparse vector $\boldsymbol{\xi}$ characterized by a relatively small number of nonzero entries corresponding to the mesh entities defining the reduced mesh $\tilde{\mathcal{E}}$.

After ECSW has computed the reduced mesh $\tilde{\mathcal{E}} \subset \mathcal{E}$, the augmented counterpart $\tilde{\mathcal{E}}^+$ is obtained by simply adding to $\tilde{\mathcal{E}}$ the neighbors to its mesh entities that participate in the same stencil of the chosen semi-discretization scheme and have not already been sampled in $\tilde{\mathcal{E}}$; and the LSPG-based HPROM associated with a LSPG-based PROM of the form given in (5) is constructed as

$$\tilde{\mathbf{r}}_n^{m+1}(\boldsymbol{\mu}) = \mathbf{0}$$

where $\tilde{\mathbf{r}}_n^{m+1}$ is the approximation of $\mathbf{r}_n^{m+1}$ (8) given in (9).

3. Construction of a nonlinear approximation manifold using a ROB and an ANN

We propose here a concept of an ANN-augmented PROM – or a PROM-ANN – that is grounded in that of an ANN-augmented affine approximation, which itself is inspired from our previous work on the development of a quadratic approximation manifold [1]. We start from the same snapshot matrix $\mathbf{S} \in \mathbb{R}^{N \times N_s}$ used to construct a traditional PROM based on the traditional affine subspace approximation. However, we aim at building a right ROB $\mathbf{V}$ whose dimension n not only verifies $n \ll N$, but is also much smaller than the dimension n_{tra} of a traditional PROM. In practice, this means that we build $\mathbf{V}$ by using a much looser tolerance ε_s in the singular value energy truncation criterion (3) than traditionally. To remedy the inaccuracy that can be expected from such a decision, we augment the corresponding affine subspace approximation with an arbitrarily nonlinear ANN approximation. Furthermore, we design the ANN approximation such that the size of the corresponding network does not scale with the large dimension N of the parametric HDM of interest, but with a much smaller dimension.

Specifically, we construct an arbitrarily nonlinear approximation manifold by approximating the solution $\mathbf{u} \in \mathbb{R}^{N \times n}$ as follows

$$\mathbf{u}(t; \boldsymbol{\mu}) \approx \tilde{\mathbf{u}}(t; \boldsymbol{\mu}) = \mathbf{u}_{\text{ref}} + \mathbf{V}\mathbf{q}(t; \boldsymbol{\mu}) + \bar{\mathbf{V}}\mathcal{N}(\mathbf{q}(t; \boldsymbol{\mu})) \quad (16)$$

where:

- We construct $\mathbf{V} \in \mathbb{R}^{N \times n}$ as usual, using the first $n \ll N$ columns of $\mathbf{U}_s$ – the matrix of left singular vectors of the snapshot matrix $\mathbf{S}$ (see Section 2.1) which spans the range of $\mathbf{S}$ – with n also much smaller than for a traditional PROM – that is, $n \ll n_{\text{tra}}$.
- We construct $\bar{\mathbf{V}} \in \mathbb{R}^{N \times \bar{n}}$ using a subset of the next $\bar{n} \ll N$ columns of $\mathbf{U}_s$, and therefore $\bar{n} \leq k - n$, where $k \leq \min(N, N_s)$ is the rank of $\mathbf{S}$ (see Section 2.1).
- Typically, we set $\bar{n} \approx n_{\text{tra}} - n$ so that $n + \bar{n} \approx n_{\text{tra}}$.
- $\mathbf{V}$ and $\bar{\mathbf{V}}$ satisfy the orthogonality properties

$$\mathbf{V}^T \mathbf{V} = \mathbf{I}_n, \quad \bar{\mathbf{V}}^T \bar{\mathbf{V}} = \mathbf{I}_{\bar{n}}, \quad \mathbf{V}^T \bar{\mathbf{V}} = \mathbf{0} \quad (17)$$

where $\mathbf{I}_n$ and $\mathbf{I}_{\bar{n}}$ denote the identity matrices of dimension n and $\bar{n}$, respectively.

- $\mathcal{N} : \mathbb{R}^n \rightarrow \mathbb{R}^{\bar{n}}$ is a map represented by an ANN whose size scales with $\max(n, \bar{n}) \ll N$.

In practice, we emphasize in (16) the contribution of the ANN and therefore choose n and $\bar{n}$ such that $n \ll \bar{n} \ll N$. This is because only the contribution of $\bar{\mathbf{V}}\mathcal{N}$ to the overall approximation (16) can effectively mitigate the Kolmogorov barrier to reducibility. The contribution of the affine subspace approximation is to provide “completeness” to the overall approximation (16).

To characterize further the ANN representation $\mathcal{N}$, we consider the projection of all solution snapshots computed and collected in the snapshot matrix $\mathbf{S}$ (or a subset of them) onto the subspaces represented by $\mathbf{V}$ and $\bar{\mathbf{V}}$. They can be written as

$$\mathbf{q}^l = \mathbf{V}^T (\mathbf{u}^l - \mathbf{u}_{\text{ref}}), \quad \bar{\mathbf{q}}^l = \bar{\mathbf{V}}^T (\mathbf{u}^l - \mathbf{u}_{\text{ref}}), \quad l = 1, \dots, N_s \quad (18)$$

Next, we apply (16) to approximate ultimately perfectly all computed solution snapshots, which yields

$$\mathbf{u}^l(\boldsymbol{\mu}) - \mathbf{u}_{\text{ref}} = \mathbf{V}\mathbf{q}^l + \bar{\mathbf{V}}\mathcal{N}(\mathbf{q}^l) \quad (19)$$

We premultiply (19) by $\bar{\mathbf{V}}^T$, which in view of the orthogonality properties (17) leads to

$$\bar{\mathbf{V}}^T (\mathbf{u}^l - \mathbf{u}_{\text{ref}}) = \mathcal{N}(\mathbf{q}^l)$$

and in view of (18) gives

$$\bar{\mathbf{q}}^l = \mathcal{N}(\mathbf{q}^l), \quad l = 1, \dots, N_s \quad (20)$$

Hence, our ANN takes as input the data set $\{\mathbf{q}^l\}_{l=1}^{N_s}$ (or a subset of it); and produces the output data set $\{\bar{\mathbf{q}}^l\}_{l=1}^{N_s}$ (or a subset of it). We determine its vector-valued hyperparameter $\boldsymbol{\eta}$ by minimizing the loss function implied by (21) – that is,

$$\boldsymbol{\eta} = \arg \min_{\boldsymbol{\eta}'} \frac{1}{N_{\text{train}}} \sum_l^{N_{\text{train}}} (\bar{\mathbf{q}}^l - \mathcal{N}(\mathbf{q}^l, \boldsymbol{\eta}'))^2 \quad (21)$$

where $N_{\text{train}} \leq N_s$ denotes the number of solution snapshots allocated to training.

4. Adaptation of LSPG and ECSW to the construction of a HPRM-ANN

4.1. Adaptation of LSPG

Given a solution approximation such as (2) or (16), LSPG is fully specified once its left ROB $\mathbf{W}$ is specified. Hence, we adapt here LSPG to the proposed concept of an ANN-augmented PROM by fully specifying the left ROB $\mathbf{W}$ associated with the approximation (16).

To this end, we first substitute in (4) the ANN-augmented affine approximation (16) for the traditional affine subspace approximation, which yields

$$\mathbf{r} \left(\underbrace{\mathbf{u}_{\text{ref}} + \mathbf{V}\mathbf{q}^{m+1}(\boldsymbol{\mu}) + \bar{\mathbf{V}}\mathcal{N}(\mathbf{q}^{m+1}(\boldsymbol{\mu}))}_{\tilde{\mathbf{u}}(\mathbf{q}^{m+1}(\boldsymbol{\mu}); \boldsymbol{\mu})}, t^{m+1}; \boldsymbol{\mu} \right) = \mathbf{0} \quad (22)$$

Next, we apply the Gauss-Newton method to solve the above overdetermined system of N equations with $n \ll N$ unknowns ($\mathbf{q}^{m+1} \in \mathbb{R}^n$), which can be summarized as follows:

- Build the sequence of iterations

$$\mathbf{q}^{m+1, \ell+1}(\boldsymbol{\mu}) = \mathbf{q}^{m+1, \ell}(\boldsymbol{\mu}) + \Delta \mathbf{q}^{m+1, \ell+1}(\boldsymbol{\mu}) \quad (23)$$

where $\mathbf{q}^{m+1, \ell}$ denotes the ℓ -th iterate generalized coordinate solution.

- Substitute (23) into (22) and perform the first-order Taylor series approximation of the result around $\mathbf{q}^{m+1, \ell}$ – which can be written as

$$\begin{aligned} \mathbf{r} \left(\tilde{\mathbf{u}}^{m+1} \left(\mathbf{q}^{m+1, \ell}(\boldsymbol{\mu}) + \Delta \mathbf{q}^{m+1, \ell+1}(\boldsymbol{\mu}); \boldsymbol{\mu} \right), t^{m+1}; \boldsymbol{\mu} \right) &\approx \mathbf{r} \left(\tilde{\mathbf{u}} \left(\mathbf{q}^{m+1, \ell}(\boldsymbol{\mu}); \boldsymbol{\mu} \right), t^{m+1}; \boldsymbol{\mu} \right) \\ &+ \mathbf{J}^{m+1, \ell}(\boldsymbol{\mu}) \frac{\partial \tilde{\mathbf{u}}}{\partial \mathbf{q}} \left(\mathbf{q}^{m+1, \ell}(\boldsymbol{\mu}) \right) \Delta \mathbf{q}^{m+1, \ell+1}(\boldsymbol{\mu}) = \mathbf{0} \end{aligned} \quad (24)$$

where

$$\mathbf{J}^{m+1, \ell}(\boldsymbol{\mu}) = \mathbf{J} \left(\tilde{\mathbf{u}} \left(\mathbf{q}^{m+1, \ell}(\boldsymbol{\mu}); \boldsymbol{\mu} \right) \right)$$

and

$$\frac{\partial \tilde{\mathbf{u}}}{\partial \mathbf{q}} \left(\mathbf{q}^{m+1, \ell}(\boldsymbol{\mu}) \right) = \frac{\partial (\mathbf{V}\mathbf{q}^{m+1, \ell}(\boldsymbol{\mu}) + \bar{\mathbf{V}}\mathcal{N}(\mathbf{q}^{m+1, \ell}(\boldsymbol{\mu})))}{\partial \mathbf{q}} = \mathbf{V} + \bar{\mathbf{V}} \frac{\partial \mathcal{N}}{\partial \mathbf{q}} \left(\mathbf{q}^{m+1, \ell}(\boldsymbol{\mu}) \right) \quad (25)$$

- Recall the approximation defined in (16), consider it at time t^{m+1} – as highlighted in (22) – then rearrange (24) as follows

$$\mathbf{W}^{m+1, \ell+1}(\boldsymbol{\mu}) \Delta \mathbf{q}^{m+1, \ell+1} = -\mathbf{r} \left(\tilde{\mathbf{u}} \left(\mathbf{q}^{m+1, \ell}(\boldsymbol{\mu}); \boldsymbol{\mu} \right), t^{m+1}; \boldsymbol{\mu} \right) \quad (26)$$

where, as in the first equality in (6), $\mathbf{W}^{m+1, \ell+1}(\boldsymbol{\mu}) = \mathbf{J}^{m+1, \ell}(\boldsymbol{\mu}) \frac{\partial \tilde{\mathbf{u}}}{\partial \mathbf{q}} \left(\mathbf{q}^{m+1, \ell}(\boldsymbol{\mu}) \right)$. Then, from (25), it follows that

$$\mathbf{W}^{m+1, \ell+1}(\boldsymbol{\mu}) = \mathbf{J}^{m+1, \ell}(\boldsymbol{\mu}) \left(\mathbf{V} + \bar{\mathbf{V}} \frac{\partial \mathcal{N}}{\partial \mathbf{q}} \left(\mathbf{q}^{m+1, \ell}(\boldsymbol{\mu}) \right) \right) \quad (27)$$

The expression of the left ROB given in (27) specifies the left ROB associated with the ANN-augmented affine approximation (16) and therefore completes the description of the LSPG method for the proposed concept of an ANN-augmented PROM.

From (6) and (27), it follows that in the case of the ANN-augmented affine approximation, the left ROB of LSPG is simply obtained by adding

$$\mathbf{W}_{\mathcal{N}}^{m+1, \ell+1}(\boldsymbol{\mu}) = \mathbf{J}^{m+1, \ell}(\boldsymbol{\mu}) \bar{\mathbf{V}} \frac{\partial \mathcal{N}}{\partial \mathbf{q}} \left(\mathbf{q}^{m+1, \ell}(\boldsymbol{\mu}) \right)$$

to the left ROB

$$\mathbf{W}_{\text{tra}}^{m+1, \ell+1}(\boldsymbol{\mu}) = \mathbf{J}^{m+1, \ell}(\boldsymbol{\mu}) \mathbf{V}$$

computed in the case of the traditional affine subspace approximation. This incurs at each Gauss-Newton iteration primarily two additional computational costs: that associated with the evaluation of the gradient $\partial\mathcal{N}/\partial\mathbf{q}$ (C_1); and that associated with the left multiplication of this gradient by the auxiliary right ROB $\bar{\mathbf{V}}$ (C_2). It also entails an additional storage requirement.

Regarding the additional cost C_1 , we reemphasize that the map $\mathcal{N} : \mathbb{R}^n \rightarrow \mathbb{R}^{\bar{n}}$ is represented by an ANN whose size scales with $\max(n, \bar{n}) \ll N$. Hence, the computational cost incurred by the online evaluation of the gradient of this map is negligible compared to that incurred by building the left ROB $\mathbf{W}_{\text{tra}}^{m+1, \ell+1}$, which grows as $\mathcal{O}(N^2 n)$. Furthermore, because by design, the proposed ANN has more outputs than inputs ($N \gg \bar{n} \gg n$), we mitigate this cost by using the *forward* rather than backward differentiation mode adopted during training – that is, by evaluating this gradient with respect to the inputs instead of the outputs.

Regarding the additional cost C_2 , it is associated with $\mathcal{O}(Nn\bar{n})$ additional operations. In the absence of hyperreduction, it is negligible compared to the cost of building $\mathbf{W}_{\text{tra}}^{m+1, \ell+1}$, which, as mentioned above, is of the order of $\mathcal{O}(N^2 n)$ operations. In the presence of hyperreduction, both costs are relatively irrelevant, particularly when the size of the reduced mesh is a tiny fraction of that of the original mesh – say, less than 1% (see Section 5.5 for further details).

As for the additional storage requirement, it is associated with storing the matrix $(\bar{\mathbf{V}}\partial\mathcal{N}/\partial\mathbf{q}) \in \mathbb{R}^{N \times n}$. Hence, it is negligible compared to the storage requirements associated with the construction of the left ROB $\mathbf{W}_{\text{tra}}^{m+1, \ell+1}$, because $n \ll N$.

Finally, we note that the standard Gauss-Newton method solves the overdetermined problem (26) by transforming it into the square problem

$$\left(\left(\mathbf{W}^{m+1, \ell+1}(\mu) \right)^T \mathbf{W}^{m+1, \ell+1}(\mu) \right) \Delta \mathbf{q}^{m+1, \ell+1} = - \left(\mathbf{W}^{m+1, \ell+1}(\mu) \right)^T \mathbf{r} \left(\tilde{\mathbf{u}} \left(\mathbf{q}^{m+1, \ell}(\mu) \right), \mathbf{t}^{m+1}; \mu \right) \quad (28)$$

and solving this problem instead. This requires however guaranteeing that problem (28) above is nonsingular – that is, ensuring that the matrix $\mathbf{W}^{m+1, \ell+1}$ has full column rank. From (27), it follows that this requirement imposes a counterpart requirement on the ANN designed for computing the map $\mathcal{N}$ whose satisfaction may not be an easy task. For this reason, problem (26) is solved instead in this work by applying the truncated SVD method to the factorization of the matrix $\mathbf{W}^{m+1, \ell+1}$, where truncation amounts in this case to regularization (for example, see [1] for more details).

4.2. Adaptation of ECSW

Next, we adapt ECSW to the concept of an ANN-augmented PROM. For this purpose, we first point the attention of the reader to (8) and the first bullet below that equation, which emphasize that for a given PROM characterized by a ROB $\mathbf{V}$, ECSW trains the cubature approximation for PROM predictions defined by generalized coordinates and not for HDM counterparts. This means that in principle, one should exercise the constructed PROM to compute and collect generalized coordinates solution snapshots, which unfortunately defeats the purpose of hyperreduction. In the case of the traditional affine subspace approximation, this issue can be elegantly resolved by simply projecting the high-dimensional solution snapshots collected in the matrix $\mathbf{S}$, or a subset of them, onto the right ROB $\mathbf{V}$. This is highlighted in (11), which describes the construction of the matrix $\mathbf{C}$ and vector $\mathbf{d}$ defining the least squares problem (14) and its early termination criterion (15).

In the case of the proposed mixed ANN-augmented affine approximation (16) however, the identification of the vector of generalized coordinates $\mathbf{q}^l$ associated with a collected high-dimensional solution snapshot $\mathbf{u}^l$ can no longer be obtained via projection onto either of the right ROB $\mathbf{V}$ or auxiliary right ROB $\bar{\mathbf{V}}$. Instead, it requires the solution of a nonlinear problem of the form

$$\delta^l(\mathbf{q}^l) = \bar{\mathbf{V}}\mathcal{N}(\mathbf{q}^l) + \mathbf{V}\mathbf{q}^l + \mathbf{u}_{\text{ref}} - \mathbf{u}^l = \mathbf{0}$$

using, for example, yet another Gauss-Newton procedure that can be summarized as follows

$$\begin{aligned} \mathbf{q}^{l,0} &= \mathbf{V}^T \mathbf{u}^l \\ \mathbf{q}^{l,\ell+1} &= \mathbf{q}^{l,\ell} - \left(\frac{\partial \delta^l}{\partial \mathbf{q}} \left(\mathbf{q}^{l,\ell} \right) \right)^+ \delta^l \left(\mathbf{q}^{l,\ell} \right) \end{aligned}$$

where the superscript ℓ designates here too the ℓ -th Gauss-Newton iteration and the superscript $+$ designates the Moore-Penrose inverse. It follows that for the same number N_h of training solution snapshots, the computational cost associated with the setup of ECSW in the case of our proposed ANN-augmented affine approximation is higher than in the case of the traditional affine subspace approximation. Nevertheless, we can justify this potential cost increase and anticipate a significant return on our investment using the following observation (also, see [1]).

Given a nonlinear problem whose PMOR is vulnerable to the Kolmogorov barrier to reducibility, we can expect the ANN-augmented affine approximation to deliver a similar level of accuracy as a traditional PROM, using however a much smaller dimension n . Considering this and the fact that the number of cubature points required for approximating a d -dimensional integral function with p cubature points along each dimension grows as p^d , we can reasonably expect ECSW to

deliver in the case of the ANN-augmented affine approximation a much smaller reduced mesh than in that of the traditional affine subspace approximation. Hence, we can expect ECSW to deliver an even better computational efficiency when it is applied to hyperreduce a PROM-ANN; and hence we can expect an ECSW-based HPROM-ANN to deliver an even better computational efficiency than a traditional HPROM. The numerical results we report in Section 5 for a popular, parametric benchmark problem support these expectations.

5. Application and performance assessment

Here, we illustrate the concept of an ANN-augmented PROM proposed in this paper using a parametric, 2D, inviscid Burgers' problem defined in a bounded computational domain Ω and a time-interval $[0, T_f]$. We also contrast the performance of this concept for this problem, after hyperreduction, with that of a traditional HPROM, for a fixed level of accuracy.

The inviscid Burgers' equation is a popular benchmark for PMOR whose semi-discretization can be written in the form given in (1). It is convection-dominated and well known for exhibiting the Kolmogorov n -width issue. Depending on the specified initial condition, boundary condition, or source term, its solution may develop propagating shocks. We focus exclusively on such conditions and source terms and consider two discrete HDMs for this equation of dimensions 125 000 and 1 125 000 that resolve the propagating shocks.

In all cases, we set $\mathbf{u}_{\text{ref}} = \mathbf{0}$. We assess the accuracy of each computed HPROM and HPROM-ANN solution time-history relative to the HDM counterpart, using the following measure of the relative error

$$\text{RE}(\boldsymbol{\mu}) = \frac{\sum_{m=0}^{N_t} \|\mathbf{u}^m(\boldsymbol{\mu}) - \tilde{\mathbf{u}}^m(\boldsymbol{\mu})\|_2}{\sum_{m=0}^{N_t} \|\mathbf{u}^m(\boldsymbol{\mu})\|_2} \quad (29)$$

where $\boldsymbol{\mu}$ and N_t denote, as before, the queried parameter point and the total number of computational time-steps (excluding the initial time-step), respectively; and $\tilde{\mathbf{u}}^m$ can be the HPROM or HPROM-ANN solution computed at time-step t^m .

We perform all computations that we report on in this section in double precision arithmetic, on a single core – except when otherwise specified – of a single node of a Linux cluster, where each node is configured with two Intel Xeon Gold 5118 processors clocked at 2.3 GHz and 192 GB of memory. Hence, all training and solution times we report are wall clock times.

5.1. Two-dimensional parametric inviscid Burgers' problem and discretization

Specifically, we consider the following parametric, 2D, inviscid Burgers' initial boundary value problem (IBVP) defined in a square and a 2D parameter domain $\mathcal{D} \subset \mathbb{R}^2$ ($N_{\mathcal{D}} = 2$)

$$\begin{aligned} \frac{\partial u_x}{\partial t} + \frac{1}{2} \left(\frac{\partial u_x^2}{\partial x} + \frac{\partial u_x u_y}{\partial y} \right) &= 0.02 \exp(\mu_2 x) \\ \frac{\partial u_y}{\partial t} + \frac{1}{2} \left(\frac{\partial u_y^2}{\partial y} + \frac{\partial u_x u_y}{\partial x} \right) &= 0 \\ u_x(0, y, t; \boldsymbol{\mu}) &= \mu_1 \\ u_x(x, y, 0) = u_y(x, y, 0) &= 1 \end{aligned} \quad (30)$$

where $x \in [0, 100]$, $y \in [0, 100]$, $t \in [0, T_f]$, $T_f = 25$, and $\boldsymbol{\mu} = (\mu_1, \mu_2) \in [4.25, 5.50] \times [0.015, 0.03]$. We note that as specified here, $\mathcal{D}$ is identical to the parameter domain chosen in [27]. However, whereas the authors of [27] considered a 1D inviscid Burgers' problem, we consider here a 2D counterpart problem.

We discretize the 2D computational domain $\Omega = [0, 100] \times [0, 100]$ using two uniform, structured meshes:

- Mesh M1. This mesh corresponds to a discretization of Ω by 250×250 equally sized "elements" (or primal computational cells). It leads to $N_e = 62\,500$ – and $N = 125\,000$.
- Mesh M2. This mesh corresponds to a discretization of Ω by 750×750 equally sized elements. It leads to $N_e = 562\,500$ – and the one order of magnitude larger dimension $N = 1\,125\,000$.

On each mesh, we semi-discretize the parametric IBVP using a *cell-centered* Godunov's method. The resulting semi-discrete HDM can be written in the form given in (1), where

$$\mathbf{u} = \begin{pmatrix} u_x \\ u_y \end{pmatrix}$$

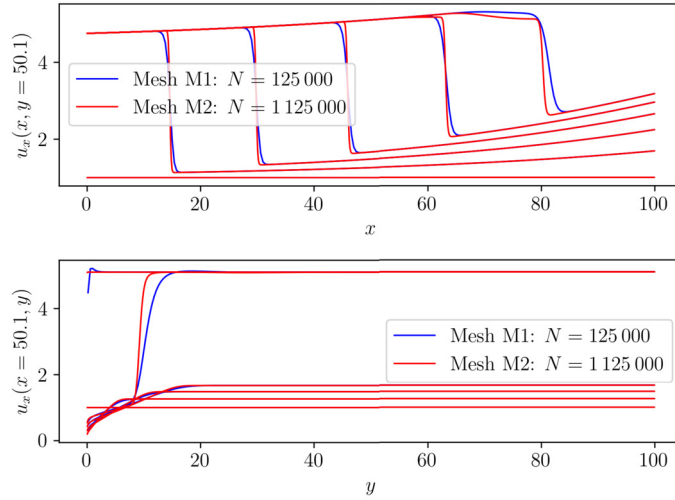


Fig. 1. Snapshots of the component u_x of the HDM solution computed at $\mu = (4.75, 0.02)$. (For interpretation of the colors in the figure(s), the reader is referred to the web version of this article.)

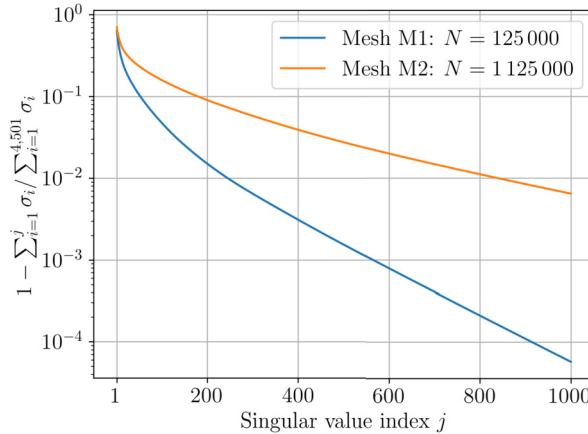


Fig. 2. Decay of the singular value energy fraction of the largest 1000 singular values of the snapshot matrix.

We discretize each resulting ordinary differential equation using the trapezoidal rule and the fixed time-step size $\Delta t = 0.05$, and therefore discretize the time-interval $]0, 25]$ in 500 equally spaced time-steps. At each time-step, we solve the resulting system of nonlinear equations using Newton's method and at each Newton iteration, we solve the resulting algebraic system of equations using a direct sparse solver.

On the single core described above, the time-to-solution of a single instance of the parametric HDM is: 718 seconds, for $N = 125\,000$ (mesh M1); and 24448 seconds – or 6.79 hours – for $N = 1\,125\,000$ (mesh M2).

Fig. 1 shows that the solution of the parametric IBVP (30) generally produces a shock that propagates from the far left to the far right of Ω ; and that the mesh M1 is underresolved compared to the mesh M2. In both cases however, the convecting structure of the captured solution leads to the Kolmogorov barrier, which manifests itself in Fig. 2 through the slow decay of the singular values of the snapshot matrix $\mathbf{S}$. In particular, the reader can observe that, at least for the IBVP considered here, the n -width issue is *significantly* aggravated when the mesh is refined.

5.2. Training the ROB

For the purpose of training the ROB $\mathbf{V}$ in both cases of a PROM and a PROM-ANN and both cases of the mesh M1 and the mesh M2, we uniformly sample the parameter domain $\mathcal{D} = [4.25, 5.50] \times [0.015, 0.03]$ by a 3×3 grid – and therefore 9 parameter points – using the constant increments $\Delta\mu_1 = 0.625$ and $\Delta\mu_2 = 0.0075$. This sampling leads to $N_s = 4501$ solution snapshots, including the initial condition which is nonparametric and therefore shared by all sampled parameter points (see (30)).

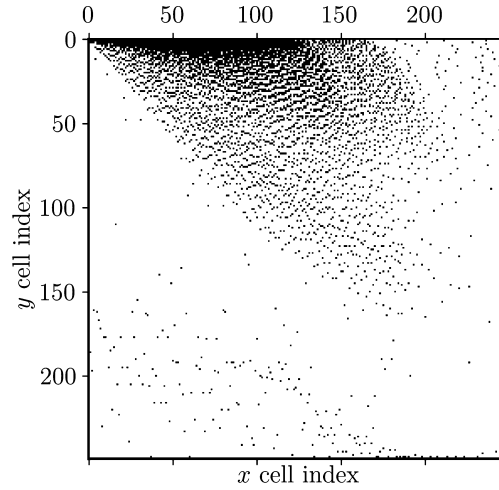


Fig. 3. ECSW-generated reduced mesh for the traditional LSPG-based PROM and the mesh M1.

5.3. Parametric nonlinear LSPG- and ECSW-based HPROM – case of the mesh M1

For the case of the mesh M1, we set ε_S^2 (3) to 5% and construct a right ROB $\mathbf{V}$ of dimension $n_{\text{tra}} = 95$, as described in Section 5.2. Using this right ROB and the corresponding left ROB $\mathbf{W}$, we train ECSW utilizing: only one of the sampled parameter points – namely, the point (4.25, 0.0225); every 10-th solution snapshot in time; and therefore a subset of $N_h = 50 (1 \times 50)$ of the collected 4501 solution snapshots. In this case, executing ECSW on the same aforementioned single core with $\varepsilon_{\text{ECSW}} = 10^{-14}$ (see (15)) delivers in 8.31 hours a reduced mesh with $n_e = 5746$ elements. Hence, this reduced mesh retains 9.19% of the $N_e = 62500$ elements of the original mesh (see Fig. 3), which explains the relatively high run time of ECSW. Indeed, for the typical, industrial-grade, RANS CFD problem, ECSW retains only 0.0029% [10] to 0.26% [1] of the total number of elements of the mesh underlying the HDM and runs much faster than reported here, even on a single core [5]. The relatively high percentage of retained elements and relatively high run time of ECSW are due in this case to the nature of the considered problem, whose solution contains a feature – namely, a shock – that travels throughout the computational domain Ω , and therefore throughout the original mesh M1.

Next, we query three different unsampled parameter points $\mu_1 = (5.19, 0.026)$, $\mu_2 = (4.56, 0.019)$, and $\mu_3 = (4.75, 0.02)$; and apply the HDM and the constructed LSPG- and ECSW-based HPROM to predict the solution of the IBVP (30) at these points, using the same fixed time-step size $\Delta t = 0.05$. We compute the maximum relative error $\mathbb{RE}_{\max}$ defined as follows

$$\mathbb{RE}_{\max} = \max_{\mu_1, \mu_2, \mu_3} \mathbb{RE}(\mu) \quad (31)$$

where the definition of $\mathbb{RE}$ is given in (29). We find that in the case of the mesh M1, $\mathbb{RE}_{\max}$ is obtained at the parameter point $\mu_1 = (5.19, 0.026)$. We report in Fig. 4 samples of the solution time-histories computed at this parameter point using the HDM and HPROM, in two near-mid-plane slices of Ω . The reader can observe that except for some occasional and small spurious oscillations, the results delivered by the HPROM are in good agreement with their counterparts delivered by the HDM. This conclusion is confirmed by the relative errors of the order of 1% reported in Table 1.

5.4. Parametric nonlinear LSPG- and ECSW-based HPROM-ANN – case of the mesh M1

Here, we first set $n = 10$ ($\ll 95$), $\bar{n} = 140$, and construct for the case of the mesh M1 a right ROB $\mathbf{V}$ and an auxiliary right ROB $\bar{\mathbf{V}}$, using the training briefly described in Section 5.2. We choose these values of n and $\bar{n}$ because for the HDM associated with the mesh M1, they turn out to lead to a HPROM-ANN that delivers an accuracy comparable to that achieved by the traditional HPROM of dimension $n_{\text{tra}} = 95$.

Next, we use the python software package PyTorch to build for the map $\mathcal{N}$ the ANN graphically depicted below

$$(n, 32) \xrightarrow{\text{ELU}} (32, 64) \xrightarrow{\text{ELU}} (64, 128) \xrightarrow{\text{ELU}} (128, 256) \xrightarrow{\text{ELU}} (256, 256) \xrightarrow{\text{ELU}} (256, \bar{n}) \quad (32)$$

This ANN has 6 linear layers of dimensions (input, output) and exponential linear unit (ELU) activation functions. We split all $N_s = 4501$ solution snapshots between training and testing. Specifically, we randomly shuffle the set of generalized coordinates associated with these snapshots and split the data 90%-10% for training-testing. On a single core of the Linux cluster that is *not a GPU*, building this ANN requires 19.3 minutes.

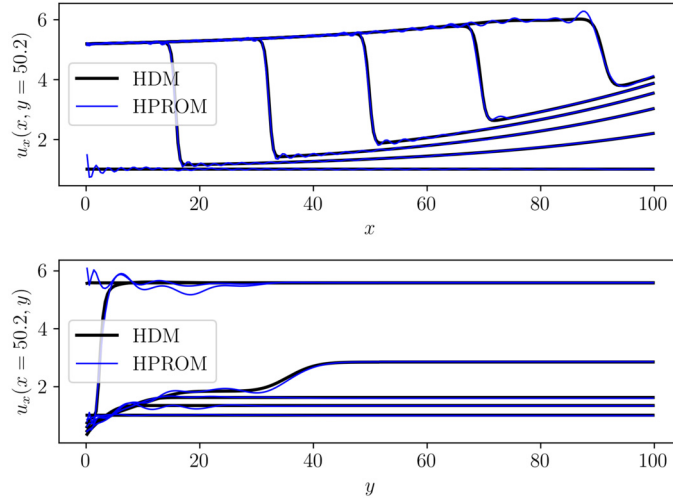


Fig. 4. Samples of the solution time-histories computed on the mesh M1 at $\mu_1 = (5.19, 0.026)$, using the HDM and the LSPG- and ECSW-based HPRM ($t = 0, 5, 10, 15, 20$, and 25): $y = 50.2$ (top); and $x = 50.2$ (bottom).

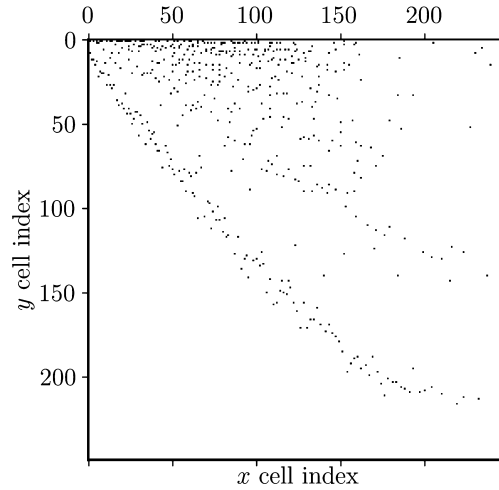


Fig. 5. ECSW-generated reduced mesh for the LSPG-based PROM-ANN and the mesh M1.

Remark 2. We also use `PyTorch` to compute online the gradient $\partial \mathcal{N} / \partial \mathbf{q}$ (forward mode automatic differentiation) whenever this gradient is needed.

Then, we use the constructed right ROB, auxiliary right ROB, and ANN to build a PROM-ANN; apply ECSW to hyperreduce it using the same approach described in Section 5.3; and transform the PROM-ANN into a HPRM-ANN. In this case, executing ECSW on the same aforementioned single core with $\varepsilon_{\text{ECSW}} = 10^{-14}$ (see (15)) delivers in 107 seconds only a reduced mesh with $n_e = 1496$ elements (see Fig. 5). Hence, in this case, ECSW delivers 280 times faster than in the case of a traditional HPRM a reduced a mesh that furthermore retains only 2.39% of the $N_e = 62500$ elements of the mesh M1; and this reduced mesh contains 3.84 times fewer elements than that delivered by ECSW for constructing the traditional HPRM (see Section 5.3). These results are consistent with the cubature interpretation of ECSW; and with the fact that the total number of cubature points required for approximating an n -dimensional integral function using n_e cubature points along each dimension grows as n_e^n .

Next, we apply the constructed LSPG- and ECSW-based HPRM-ANN at the same three unsampled parameter points as in Section 5.3 to predict the solution of the IBVP (30); and compute $\mathbb{R}\mathbb{E}_{\max}$ using the same fixed time-step size $\Delta t = 0.05$. We find that in this case, $\mathbb{R}\mathbb{E}_{\max}$ is obtained at the parameter point $\mu_2 = (4.56, 0.019)$. We report in Fig. 6 samples of the solution time-histories computed at this parameter point using the HDM and HPRM, in two same near-mid-plane slices of Ω as in Fig. 4. The reader can observe a good agreement between the results obtained using the HPRM-ANN and their HDM counterparts, which is again confirmed by the relative errors of 1% reported in Table 1. More importantly however, the comparison of the results displayed in Fig. 6 and Fig. 4 highlights the superior performance of the HPRM-ANN at minimizing spurious oscillations around the traveling shock and attenuating their amplitudes.

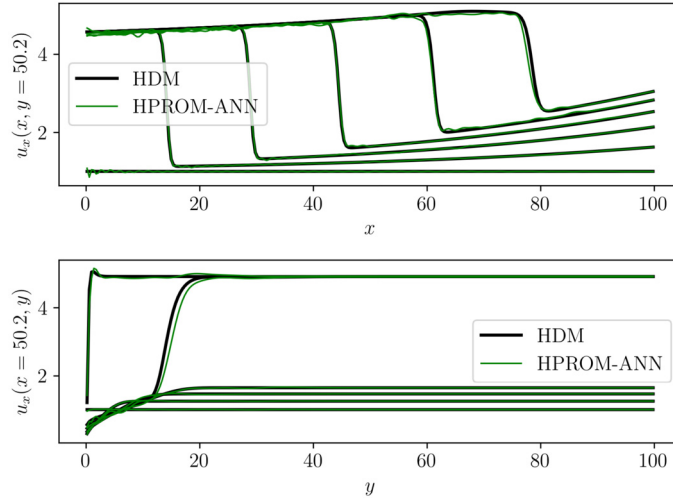


Fig. 6. Samples of the solution time-histories computed on the mesh M1 at $\mu_2 = (4.56, 0.019)$ using the HDM and the LSPG- and ECSW-based HPROM-ANN ($t = 0, 5, 10, 15, 20$, and 25): $y = 50.2$ (top); and $x = 50.2$ (bottom).

Table 1

Model parameters and offline/online performance results on a single core for the case of the mesh M1.

Computational model	n (N for HDM)	n_e (N_e for HDM)	$\mathbb{R}\mathbb{E}_{\max}$	Time (minutes)	Speedup factor
HDM ($\Delta t = 0.05$)	125 000	62 500	—	11.97	—
HPROM (ECSW, offline)	95	5 746	—	499	—
HPROM (online, $\Delta t = 0.05$)	95	5 746	1.50%	1.57	7.62
HPROM-ANN (ANN, offline)	10	—	—	19.30	—
HPROM-ANN (ECSW, offline)	10	1 496	—	1.79	—
HPROM-ANN (online, $\Delta t = 0.05$)	10	1 496	1.51%	0.23	52.04

5.5. Performance comparisons for the case of the mesh M1

Table 1 summarizes the various models and model parameters considered for the solution of problem (30) on the mesh M1; and the offline as well as online performances of these models on the single core described at the beginning of Section 5. The results reported in this table suggest that, given a set of pre-computed solution snapshots, the construction offline of the HPROM-ANN is computationally tractable. They also demonstrate that the online performance of the HPROM-ANN is superior to that of the traditional HPROM. Specifically, these results show that for the case of the HDM of dimension $N = 125\,000$:

- The training of the ANN part of the HPROM-ANN on a single core that is not a GPU requires only 19.30 minutes.
- For fixed model order reduction and hyperreduction precisions, ECSW computes a reduced mesh for constructing the HPROM-ANN 280 times faster than it computes a counterpart reduced mesh for constructing the traditional HPROM; and this mesh contains 3.84 times fewer elements than its counterpart associated with the traditional HPROM.
- The HPROM-ANN can deliver the same desirable level of accuracy as the traditional HPROM, using however an order of magnitude smaller dimension and almost seven times faster.
- The HPROM-ANN can reproduce a convection-and-shock-dominated HDM solution time-history with 98.49% accuracy, using however a four orders of magnitude smaller dimension and 52 times faster.

Remark 3. The following remark concerns the relationship between the mesh reduction factor N_e/n_e delivered by ECSW and the factor by which a HPROM accelerates a PROM.

The run time of a nonlinear, implicit, HDM is typically dominated by the computational cost associated with the solution at each Newton iteration, within each time-step, of a sparse system of linear equations of size N . If this system is solved using a direct sparse solver, the aforementioned cost has the complexity $O(N \text{nz}(\mathbf{J})^\beta)$, where $\text{nz}(\mathbf{J})$ denotes the number of nonzero coefficients of the Jacobian of the residual $\mathbf{r}$ with respect to the solution $\mathbf{u}$ and β is some positive constant. If this system is solved however using a Krylov-based iterative method such as a preconditioned GMRES method, the corresponding cost has a complexity that depends on many factors, including the chosen preconditioner and the allowed maximum number of iterations. For simplicity, we assume here that in both cases, the complexity of the solution of the linear system of

equations can be approximated as $C_{\text{sol}}^{\text{HDM}} = \mathcal{O}(c_{\text{sol}} N)$, where c_{sol} is some positive constant that depends on the chosen algebraic solver. For realistic systems, one can also reasonably assume that $N \approx \gamma_1 \gamma_2 N_e$, where γ_2 is the ratio of the number of nodes and number of elements of the given mesh; and γ_1 denotes the number of unknowns defined at each node, which, for the application considered herein, is constant. In this case, the aforementioned solution cost can be estimated as $C_{\text{sol}}^{\text{HDM}} = \mathcal{O}(c_{\text{sol}} N_e)$.

On the other hand, the run time of a nonlinear, implicit, LSPG-based HPROM or HPROM-ANN is dominated by the computational costs associated with forming and solving at each Gauss-Newton iteration, within each time-step, the system of equations (28). For realistic systems (see above), these costs can be characterized as follows:

- From the complexity analysis outlined in Section 4.1, it follows that for a HPROM, the cost C_{for} associated with forming the system of equations (28) has the complexity $C_{\text{for}} = \mathcal{O}(n_e^3 n)$; and for a HPROM-ANN, it has the complexity $C_{\text{for}} = \mathcal{O}(n_e^3 n) + \mathcal{O}(n_e n \bar{n})$.
- Assuming that $n_e \gg n$ and $n_e \gg \bar{n}$, which is reasonable for realistic problems, the cost C_{sol} associated with solving the system of equations (28) using the truncated SVD method has the complexity $C_{\text{sol}} = \mathcal{O}(n_e n^3)$.

From the above simplified complexity analysis, it follows that when using the HPROM or the HPROM-ANN to predict the solution of a realistic problem of interest, the following speedup factors can be expected

$$\text{SF}_{\text{HPROM}} = \frac{\mathcal{O}(c_{\text{sol}} N_e)}{\mathcal{O}(n_e^3 n) + \mathcal{O}(n_e n^3)} \approx \frac{\mathcal{O}(c_{\text{sol}} N_e)}{\mathcal{O}(n_e^3 n)} \quad (33)$$

and

$$\text{SF}_{\text{HPROM-ANN}} = \frac{\mathcal{O}(c_{\text{sol}} N_e)}{\mathcal{O}(n_e^3 n) + \mathcal{O}(n_e n \bar{n}) + \mathcal{O}(n_e n^3)} \approx \frac{\mathcal{O}(c_{\text{sol}} N_e)}{\mathcal{O}(n_e^3 n)} \quad (34)$$

Hence, in both cases, the estimation of the expected speedup factor is the same, but neither n nor n_e is the same. In particular, (34) shows that for realistic problems, the speedup factor that can be expected from the HPROM-ANN is asymptotically independent from the parameter $\bar{n}$ of this computational model. Furthermore, both (33) and (34) reveal that $\text{SF} \geq \mathcal{O}(N_e/n_e)$, if $n_e \leq \sqrt{c_{\text{sol}}/\bar{n}}$ – that is, if for the specified tolerance $\varepsilon_{\text{ECSW}}$ (see (15)), ECSW sufficiently reduces the size of the original mesh underlying the HDM; and $\text{SF} \leq \mathcal{O}(N_e/n_e)$ otherwise – that is, if for the same tolerance $\varepsilon_{\text{ECSW}}$, $n_e \geq \sqrt{c_{\text{sol}}/\bar{n}}$, and thus ECSW does not sufficiently reduce the size of the original mesh. These results explain the following additional noteworthy observation in Table 1. Indeed, the reader can note in this table that the HPROM accelerates the online run time of the HDM by a factor of 7.62, which is lesser than the mesh reduction factor $(N_e/n_e)_{\text{HPROM}} = 10.88$; but the HPROM-ANN, which compresses further the size of the reduced mesh by a factor equal to 3.84, accelerates the online run time of the HDM by a factor of 52.04, which is greater than the mesh reduction factor $(N_e/n_e)_{\text{HPROM-ANN}} = 41.78$.

5.6. Performance comparisons for the case of the mesh M2

In Section 5.5, we have considered the HDM associated with the mesh M1 and reported performance results for a corresponding HPROM-ANN that demonstrate the superior online performance of the latter computational model over that of a traditional HPROM; and suggest the computational tractability of its offline stage – and thus, of its ANN component – given a set of pre-computed solution snapshots. Here, we consider the HDM associated with the mesh M2, whose dimension $N = 1\,125\,000$ is an order of magnitude larger than that of its counterpart associated with the mesh M1, and focus on demonstrating the suggested computational tractability of the offline stage of our HPROM-ANN concept. Furthermore, we demonstrate the significance of hyperreduction even when the PROM of interest is enriched by the ANN component that we propose. For these reasons, we consider here, for the HDM associated with the mesh M2, a PROM-ANN and two corresponding HPROM-ANNs characterized by two different configurations $(n, \bar{n})$.

First, we consider the same configuration $(n, \bar{n}) = (10, 140)$ as in the case of the mesh M1 and build the corresponding ANN (32) using PyTorch. We refer to this configuration as Option 1. As before, we split all $N_s = 4\,501$ pre-computed solution snapshots between training and testing; randomly shuffle the set of generalized coordinates associated with these snapshots; and split them 90%-10% for training-testing.

Next, we consider the alternative configuration $(n, \bar{n}) = (20, 280)$, which we refer to as Option 2. For the sake of experimentation, we build for this configuration an ANN with a *single* hidden layer of size $n \times \bar{n}$. We train this ANN using otherwise the same settings and amounts of data as in the case of the mesh M1 and the previous ANN built for mesh M2, which takes 20.80 minutes of wall clock time.

Then, we hyperreduce both resulting PROM-ANNs corresponding to Option 1 and Option 2 using ECSW; and 1 core and 24 cores of the same Linux cluster, respectively. For this purpose, we set $\varepsilon_{\text{ECSW}} = 10^{-14}$ (see (15)) and train ECSW as in Section 5.3 and Section 5.4 – that is, using only the sampled parameter point (4.25, 0.0225); every 10-th solution snapshot in time; and therefore a subset of $N_h = 50(1 \times 50)$ of the collected 4501 solution snapshots.

We report in Table 2 the performance results obtained for both offline and online stages of all considered computational models and compare them to their counterparts for a traditional HPROM of dimension $n_{\text{tra}} = 95$.

Table 2

Model parameters and offline/online performance results on a single core (except when otherwise specified) for the case of the mesh M2.

Computational model	n (N for HDM)	n_e (N_e for HDM)	$\mathbb{RE}_{\max}$	Time (minutes)	Speedup factor
HDM ($\Delta t = 0.05$)	1 125 000	562 500	—	407.47	—
HPROM (ECSW, offline, 24 cores)	95	63 106	—	45.25	—
HPROM (online, $\Delta t = 0.05$)	95	63 106	8.05%	35.08	11.61
PROM-ANN (ANN, offline)	10	—	—	23.40	—
PROM-ANN (online, $\Delta t = 0.05$)	10	—	5.50%	23.90	17.05
HPROM-ANN (ANN, offline)	10	—	—	23.40	—
HPROM-ANN (ECSW, offline)	10	3 496	—	26.40	—
HPROM-ANN (online, $\Delta t = 0.05$)	10	3 496	3.43%	0.500	814.94
PROM-ANN (ANN, offline)	20	—	—	20.80	—
PROM-ANN (online, $\Delta t = 0.05$)	20	—	4.60%	65.13	6.26
HPROM-ANN (ANN, offline)	20	—	—	20.80	—
HPROM-ANN (ECSW, offline, 24 cores)	20	22 984	—	5.21	—
HPROM-ANN (online, $\Delta t = 0.05$)	20	22 984	4.72%	8.74	46.62

Based on the results reported in Table 2, we make the following noteworthy observations:

- For both Option 1 and Option 2, the training times of the ANN part of the HPROM-ANN – namely, 23.40 minutes for Option 1 and 20.80 minutes for Option 2 – are close to the training time of the ANN built in the case of the much coarser mesh M1 (19.30 minutes, see Section 5.4). This demonstrates the computational tractability of the ANN part of our PROM-ANN concept for large-dimensional problems (recall that here, $N = 1\,125\,000$). We note that this is not surprising, considering that the complexity of the training of the ANN component of a PROM-ANN is independent of the large dimension N of the HDM: it depends only on the much lower dimensions n and $\bar{n}$.
- For both Option 1 and Option 2, the training times of ECSW – namely, 26.40 minutes for Option 1 and 5.21 minutes for Option 2 – demonstrate the computational tractability of the hyperreduction part of our HPROM-ANN concept for large-scale meshes. In particular, the training of ECSW for Option 2 is 8.69 times faster than that for the traditional HPROM.
- From both observations above, it follows that the offline stage of our HPROM-ANN concept is computationally tractable (given a set of pre-computed solution snapshots).
- For each of Option 1 and Option 2, the corresponding HPROM-ANN achieves a good accuracy as measured by $\mathbb{RE}_{\max}$ (31), especially considering the important contributions of relatively small shock location errors to this two-norm-based measure of the relative error. These contributions explain why $\mathbb{RE}_{\max}$ is slightly larger for the larger configuration $(n, \bar{n}) = (20, 280)$; however, close inspections of Fig. 7 and Fig. 8 reveal that the solution predicted by the HPROM-ANN characterized by the larger configuration $(n, \bar{n}) = (20, 280)$ (Option 2) is marred by fewer spurious oscillations of smaller amplitudes than the counterpart solution predicted by the HPROM-ANN characterized by the smaller configuration $(n, \bar{n}) = (10, 140)$ (Option 1).
- The HPROM-ANN characterized by the smaller configuration $(n, \bar{n}) = (10, 140)$ delivers an impressive wall clock time speedup factor of roughly 815 with respect to the HDM; and a wall clock time speedup factor of 47.80 with respect to the corresponding PROM-ANN.
- The HPROM-ANN characterized by the larger configuration $(n, \bar{n}) = (20, 280)$ delivers a wall clock time speedup factor of 46.62 with respect to the HDM; and a wall clock time speedup factor of 7.45 with respect to the corresponding PROM-ANN.
- From both observations above, it follows that hyperreduction is essential even for a PROM-ANN.
- The HPROM-ANNs characterized by the smaller and larger configurations $(n, \bar{n}) = (10, 140)$ and $(n, \bar{n}) = (20, 280)$, respectively, deliver wall clock time speedup factors of roughly 70 and 4 with respect to the considered traditional HPROM, respectively, while delivering a better level of accuracy.

5.7. Effects of variations in n and $\bar{n}$ on the performance of a HPROM-ANN

The automation of the selection of n and $\bar{n}$ for the efficient PMOR of a HDM known to exhibit the n -width issue is a research topic that we are currently pursuing: hence, it is beyond the scope of this paper. Nevertheless, to pave the way for this development, we perform here a parametric study aimed at delivering insight into the effects of variations in n and $\bar{n}$ on the accuracy and performance of a HPROM-ANN constructed for the same HDM described above for the IBVP (30). To this end, we parameterize the variations of n and $\bar{n}$ using two parameters, n_{tra} and α , and the following governing relations

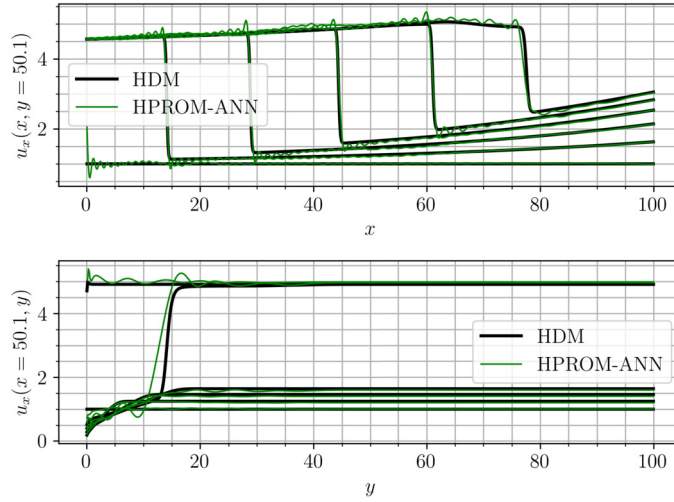


Fig. 7. Samples of the solution time-histories computed on the mesh M2 at $\mu_2 = (4.56, 0.019)$, using the HDM and the LSPG- and ECSW-based HPROM-ANN characterized by $(n, \bar{n}) = (10, 140)$ ($t = 0, 5, 10, 15, 20$, and 25): $y = 50.1$ (top); and $x = 50.1$ (bottom).

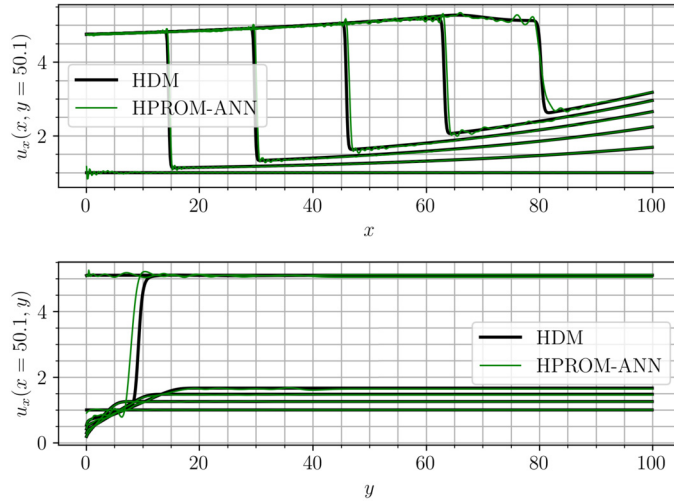


Fig. 8. Samples of the solution time-histories computed on the mesh M2 at $\mu_2 = (4.75, 0.02)$, using the HDM and the LSPG- and ECSW-based HPROM-ANN characterized by $(n, \bar{n}) = (20, 280)$ ($t = 0, 5, 10, 15, 20$, and 25): $y = 50.1$ (top); and $x = 50.1$ (bottom).

$$n + \bar{n} = n_{\text{tra}}, \text{ where } n_{\text{tra}} \in [n_{\text{low}}, n_{\text{upp}}] \subset \mathbb{N} \quad \text{and} \quad n = \alpha \bar{n}, \text{ where } \alpha \in [0, 1] \subset \mathbb{R} \quad (35)$$

In (35), the first parameterization guideline is consistent with the third bullet in Section 3. In this guideline, n_{tra} denotes the minimum between the dimension required for a PROM or HPROM to mitigate the Kolmogorov n -width issue – that is, to achieve a desired level of approximation accuracy – and the maximum dimension of a PROM or HPROM that is computationally affordable. In the second guideline within (35), the range of the real-valued scalar α is chosen such that the corresponding range of n includes the choice $n \ll \bar{n} \ll N$ made in Section 3; and goes beyond it, for the purpose of error trend analysis.

For simplicity, but without any loss of generality, we focus on the HDM associated with mesh M1. Hence, we set n_{low} to $n_{\text{low}} = 95$, in which case $(n, \bar{n}) = (95, 0)$ corresponds to the traditional HPROM considered in Section 5.3 and shown to deliver an accuracy level of 98.50%; and set n_{upp} to $n_{\text{upp}} = 150$.

We consider the same parameter domain $\mathcal{D}$, the same spatial and temporal discretizations, and the same fixed time-step as those considered in the previous subsections of Section 5. We train the ROB $\mathbf{V}$ as before – that is, as described in Section 5.2; and build the ANN component of each PROM-ANN we construct, and hyperreduce the resulting PROM-ANN, using the same approaches and tools as before – that is, as described in Section 5.4.

We sample the ranges of n_{tra} and α given in (35) to generate several $(n, \bar{n})$ configurations. For each such configuration, we construct a HPROM-ANN $(n, \bar{n})$; apply it to predict the solution of the IBVP (30) at the same 3 unsampled parameter points as in Section 5.3; and compute the maximum relative error $\mathbb{RE}_{\text{max}}$ (31).

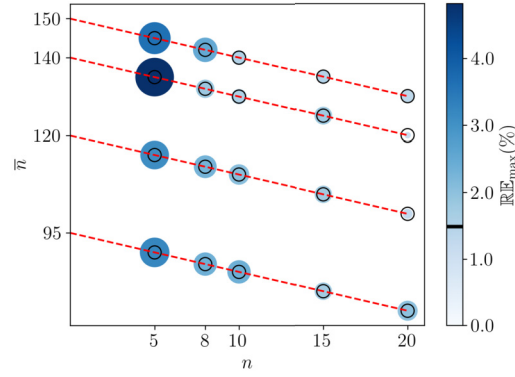


Fig. 9. Variations of the maximum relative error with n_{tra} , n , and $\bar{n}$.

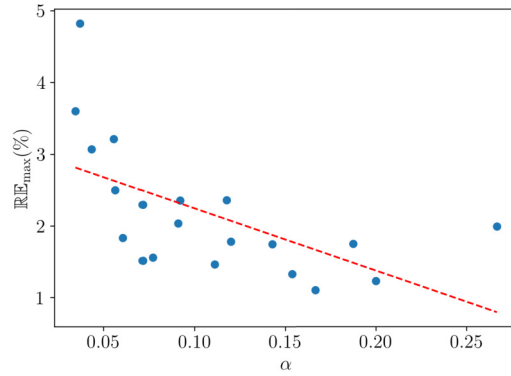


Fig. 10. Variations of the maximum relative error with α .

We report in Fig. 9 the variations of RE_{max} with n , $\bar{n}$, and $n + \bar{n} = n_{\text{tra}}$. These variations are displayed using two different graphical means: shades of blue, with lighter and darker shades corresponding to smaller and larger relative errors, respectively; and disks of different sizes, with smaller and larger radii corresponding to smaller and larger relative errors, respectively. In this figure, each red dashed line corresponds to a straight line $n + \bar{n} = n_{\text{tra}}$ and therefore to a different value of the parameter n_{tra} ; and the black circles associated with the aforementioned disks and the horizontal black line in the color bar mark the maximum relative error of 1.50% obtained for the nominal HPROM discussed in Section 5.3.

Focusing in Fig. 9 on the straight lines, the reader can observe that:

- For $n = 5$, increasing the value of $\bar{n}$ by increasing the value of n_{tra} does not reduce the maximum relative error significantly. For $n \geq 8$ however, increasing the value of $\bar{n}$ by increasing the value of n_{tra} reduces the maximum relative error. While these results highlight the benefit of $\bar{n}$ and increasing its value, they also suggest the existence of a minimum value of n below which reducing the error of the HPROM using an ANN component is difficult. There are a few configurations $(n, \bar{n})$ that do not follow this trend however: we believe the reason to be an imperfect training of the ANN at these configurations.
- When n is increased, the maximum relative error is typically decreased.
- The smallest maximum relative error $\text{RE}_{\text{max}} = 1.10\%$ is achieved for $(n, \bar{n}) = (20, 120)$. In this configuration however, the HPROM-ANN delivers the solution of the IBVP (30) in 0.30 minute, while for $(n, \bar{n}) = (10, 140)$, it delivers in 0.23 minutes a solution with $\text{RE}_{\text{max}} = 1.51\%$. These results highlight the accuracy versus speed trade-off governed by the choice of the configuration $(n, \bar{n})$ of a HPROM-ANN.

The reader can also observe that in all cases however, the maximum relative error varies between 1.10% and 4.82%. For $n \geq 8$ however, RE_{max} varies between 1.10% and 2.55%. These observations suggest that an automated method for selecting the optimal values of n and $\bar{n}$ from the accuracy viewpoint can afford a reasonable error margin without affecting engineering accuracy.

We report in Fig. 10 the variations of RE_{max} with the parameter α , when n_{tra} is varied in $[95, 150]$, using the same graphical means as in Fig. 9. Here, the red dashed line is a least squares regression line that approximates the relationship between RE_{max} and α in the scatterplot. As expected, the maximum relative error is decreased when α is increased – that is, when n is increased.

6. Conclusions

The concept of a projection-based reduced-order model (PROM) augmented by an artificial neural network (ANN) – or a PROM-ANN – we propose in this paper distinguishes itself from many previous attempts at exploiting ANNs in model order reduction in two major aspects: 1) the size of its neural network does not scale with the large dimension N of the underlying high-dimensional model (HDM), but with a much lower dimension $\bar{n} \ll N$ related to the dimension $n \ll N$ of the PROM part of this concept; and 2) it is hyperreducible to a HPROM-ANN using readily available hyperreduction methods. Consequently, our concept is computationally tractable offline, including for very-large scale computational models. While it is inspired from our previous work on polynomial approximation manifolds [1], it also distinguishes itself from that work in two other aspects: a) it enriches the traditional affine subspace approximation a priori rather than a posteriori; and b) it *explicitly* computes the enrichment using $\bar{n}$ left singular vectors truncated during the construction of the traditional reduced-order basis via the singular value decomposition of the snapshot matrix.

For a parametric, shock-dominated benchmark problem that exhibits the Kolmogorov n -width issue and characterized by $N = 1\,125\,000$, and for the settings $n = 10$ and $\bar{n} = 140$, the offline training of the ANN part of our PROM-ANN concept requires 23.40 minutes wall clock time on a single core of a Linux cluster that is not a GPU. The offline construction on the same single core of a reduced mesh for the HPROM-ANN counterpart requires 26.40 minutes wall clock time.

For the same benchmark problem, the constructed HPROM-ANN delivers a better level of accuracy than that delivered by a traditional HPROM of dimension $n_{\text{tra}} = 95$ – using an almost order of magnitude smaller dimension. Consequently, this HPROM-ANN executes roughly 70 times faster online than its counterpart traditional HPROM.

When compared to the HDM, the aforementioned HPROM-ANN, whose dimension $n = 10$ is five orders of magnitude smaller than the dimension $N = 1\,125\,000$ of the HDM, reproduces the HDM solution almost 815 times faster and with almost 3% accuracy.

In summary, our proposed concept of a HPROM-ANN has the potential for mitigating the Kolmogorov reducibility barrier. We are currently working on automating the setting of its parameters n and $\bar{n}$, and on confirming as well as quantifying its potential for industry-relevant, parametric, convection-dominated, turbulent flow problems and highly nonlinear structural dynamics problems.

CRedit authorship contribution statement

Joshua Barnett: Conceptualization, Formal analysis, Investigation, Methodology, Software, Validation, Visualization, Writing – original draft. **Charbel Farhat:** Conceptualization, Formal analysis, Funding acquisition, Resources, Supervision, Writing – review & editing. **Yvon Maday:** Conceptualization, Formal analysis, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgements

The authors acknowledge the support by the Air Force Office of Scientific Research under Grant FA9550-20-1-0358 and Grant FA9550-22-1-0004.

References

- [1] Joshua Barnett, Charbel Farhat, Quadratic approximation manifold for mitigating the Kolmogorov barrier in nonlinear projection-based model order reduction, *J. Comput. Phys.* 464 (2022) 111348.
- [2] Matthew J. Zahr, Philip Avery, Charbel Farhat, A multilevel projection-based model order reduction framework for nonlinear dynamic multiscale problems in structural and solid mechanics, *Int. J. Numer. Methods Eng.* 112 (8) (2017) 855–881.
- [3] Wanli He, Philip Avery, Charbel Farhat, In situ adaptive reduction of nonlinear multiscale structural dynamics models, *Int. J. Numer. Methods Eng.* 121 (22) (2020) 4971–4988.
- [4] Gianluigi Rozza, Dinh Bao Phuong Huynh, Anthony T. Patera, Reduced basis approximation and a posteriori error estimation for affinely parametrized elliptic coercive partial differential equations, *Arch. Comput. Methods Eng.* 15 (3) (2008) 229–275.
- [5] Charbel Farhat, Philip Avery, Todd Chapman, Julien Cortial, Dimensional reduction of nonlinear finite element dynamic models with finite rotations and energy-based mesh sampling and weighting for computational efficiency, *Int. J. Numer. Methods Eng.* 98 (9) (2014) 625–662.
- [6] Herbert Egger, Thomas Kugler, Björn Liljegren-Sailer, Nicole Marheineke, Volker Mehrmann, On structure-preserving model reduction for damped wave propagation in transport networks, *SIAM J. Sci. Comput.* 40 (1) (2018) A331–A365.
- [7] Charbel Farhat, Sebastian Grimberg, Andrea Manzoni, Alfio Quarteroni, Computational Bottlenecks for Proms: Precomputation and Hyperreduction. *Model Order Reduction*, De Gruyter, Berlin, 2020, pp. 181–244.

- [8] Jeffrey P. Thomas, Earl H. Dowell, Kenneth C. Hall, Three-dimensional transonic aeroelasticity using proper orthogonal decomposition-based reduced-order models, *J. Aircr.* 40 (3) (2003) 544–551.
- [9] David Amsallem, Charbel Farhat, Interpolation method for adapting reduced-order models and application to aeroelasticity, *AIAA J.* 46 (7) (2008) 1803–1813.
- [10] Sebastian Grimberg, Charbel Farhat, Radek Tezaur, Charbel Bou-Mosleh, Mesh sampling and weighting for the hyperreduction of nonlinear Petrov–Galerkin reduced-order models with local reduced-order bases, *Int. J. Numer. Methods Eng.* 122 (7) (2021) 1846–1874.
- [11] Sebastian Grimberg, Charbel Farhat, Noah Youkilis, On the stability of projection-based model order reduction for convection-dominated laminar and turbulent flows, *J. Comput. Phys.* 419 (2020) 109681.
- [12] Radek Tezaur, Faisal As'ad, Charbel Farhat, Robust and globally efficient reduction of parametric, highly nonlinear computational models and real time online performance, *Comput. Methods Appl. Mech. Eng.* 399 (2022) 115392.
- [13] Lambert Fick, Yvon Maday, Anthony T. Patera, Tommaso Taddei, A stabilized pod model for turbulent flows over a range of Reynolds numbers: optimal parameter sampling and constrained projection, *J. Comput. Phys.* 371 (2018) 214–243.
- [14] Constantin Greif, Karsten Urban, Decay of the Kolmogorov n -width for wave problems, *Appl. Math. Lett.* 96 (2019) 216–222.
- [15] David Amsallem, Matthew J. Zahr, Charbel Farhat, Nonlinear model order reduction based on local reduced-order bases, *Int. J. Numer. Methods Eng.* 92 (10) (2012) 891–916.
- [16] Melissa B. Rivers, Ashley Dittberner, Experimental investigations of the NASA common research model, *J. Aircr.* 51 (4) (2014) 1183–1193.
- [17] K.M. Washabaugh, M.J. Zahr, C. Farhat, On the use of discrete nonlinear reduced-order models for the prediction of steady-state flows past parametrically deformed complex geometries, in: 54th AIAA Aerosp. Sci. Meet, San Diego, California, 2016.
- [18] Gerrit Welper, Interpolation of functions with parameter dependent jumps by transformed snapshots, *SIAM J. Sci. Comput.* 39 (4) (2017) A1225–A1250.
- [19] Nicolas Cagniat, Yvon Maday, Benjamin Stamm, Model order reduction for problems with large convection effects, in: *Contributions to Partial Differential Equations and Applications*, Springer, 2019, pp. 131–150.
- [20] Felix Black, Philipp Schulze, Benjamin Unger, Projection-based model reduction with dynamically transformed modes, *ESAIM: Math. Model. Numer. Anal.* 54 (6) (2020) 2011–2043.
- [21] Rambod Mojjani, Maciej Balajewicz, Physics-aware registration based auto-encoder for convection dominated PDEs, *arXiv preprint, arXiv:2006.15655*, 2020.
- [22] Julius Reiss, Optimization-based modal decomposition for systems with multiple transports, *SIAM J. Sci. Comput.* 43 (3) (2021) A2079–A2101.
- [23] Andrea Ferrero, Tommaso Taddei, Lei Zhang, Registration-based model reduction of parameterized two-dimensional conservation laws, *J. Comput. Phys.* 457 (2022) 111068.
- [24] Marzieh Alireza Mirhoseini, Matthew J. Zahr, Model reduction of convection-dominated partial differential equations via optimization-based implicit feature tracking, *J. Comput. Phys.* (2022) 111739.
- [25] Shobhit Jain, Paolo Tiso, Johannes B. Rutzmoser, Daniel J. Rixen, A quadratic manifold for model order reduction of nonlinear structural dynamics, *Comput. Struct.* 188 (2017) 80–94.
- [26] Rudy Geelen, Stephen Wright, Karen Willcox, Operator inference for non-intrusive model reduction with quadratic manifolds, *Comput. Methods Appl. Mech. Eng.* 403 (2023) 115717.
- [27] Kookjin Lee, Kevin T. Carlberg, Model reduction of dynamical systems on nonlinear manifolds using deep convolutional autoencoders, *J. Comput. Phys.* 404 (2020) 108973.
- [28] Stefania Fresca, Luca Dede, Andrea Manzoni, A comprehensive deep learning-based approach to reduced order modeling of nonlinear time-dependent parametrized PDEs, *J. Sci. Comput.* 87 (2) (2021) 1–36.
- [29] Anthony Gruber, Max Gunzburger, Lili Ju, Zhu Wang, A comparison of neural network architectures for data-driven reduced-order modeling, *Comput. Methods Appl. Mech. Eng.* 393 (2022) 114764.
- [30] Davide Papapicco, Nicola Demo, Michele Girfoglio, Giovanni Stabile, Gianluigi Rozza, The neural network shifted-proper orthogonal decomposition: a machine learning approach for non-linear reduction of hyperbolic equations, *Comput. Methods Appl. Mech. Eng.* 392 (2022) 114687.
- [31] Zhichao Peng, Min Wang, Fengyan Li, A learning-based projection method for model order reduction of transport problems, *J. Comput. Appl. Math.* 418 (2023) 114560.
- [32] Shady E. Ahmed, Omer San, Adil Rasheed, Traian Iliescu, A long short-term memory embedding for hybrid uplifted reduced order models, *Phys. D: Nonlinear Phenom.* 409 (2020) 132471.
- [33] Stefania Fresca, Andrea Manzoni, POD-DL-ROM: enhancing deep learning-based reduced order models for nonlinear parametrized PDEs by proper orthogonal decomposition, *Comput. Methods Appl. Mech. Eng.* 388 (2022) 114181.
- [34] Omer San, Romit Maulik, Neural network closures for nonlinear model order reduction, *Adv. Comput. Math.* 44 (6) (2018) 1717–1750.
- [35] Qian Wang, Nicolò Ripamonti, Jan S. Hesthaven, Recurrent neural network closure of parametric POD–Galerkin reduced-order models based on the Mori–Zwanzig formalism, *J. Comput. Phys.* 410 (2020) 109402.
- [36] Youngkyu Kim, Youngsoo Choi, David Widemann, Tarek Zohdi, A fast and accurate physics-informed neural network reduced order model with shallow masked autoencoder, *J. Comput. Phys.* 451 (2022) 110841.
- [37] Kevin Carlberg, Charbel Bou-Mosleh, Charbel Farhat, Efficient non-linear model reduction via a least-squares Petrov–Galerkin projection and compressive tensor approximations, *Int. J. Numer. Methods Eng.* 86 (2) (2011) 155–181.
- [38] Kevin Carlberg, Charbel Farhat, Julien Cortial, David Amsallem, The GNAT method for nonlinear model reduction: effective implementation and application to computational fluid dynamics and turbulent flows, *J. Comput. Phys.* 242 (2013) 623–647.
- [39] Maxime Barrault, Yvon Maday, Ngoc Cuong Nguyen, Anthony T. Patera, An ‘empirical interpolation’ method: application to efficient reduced-basis discretization of partial differential equations, *C. R. Math.* 339 (9) (2004) 667–672.
- [40] Saifon Chaturantabut, Danny C. Sorensen, Nonlinear model reduction via discrete empirical interpolation, *SIAM J. Sci. Comput.* 32 (5) (2010) 2737–2764.
- [41] David Ryckelynck, A priori hyperreduction method: an adaptive approach, *J. Comput. Phys.* 202 (1) (2005) 346–366.
- [42] Charbel Farhat, Todd Chapman, Philip Avery, Structure-preserving, stability, and accuracy properties of the energy-conserving sampling and weighting method for the hyper reduction of nonlinear finite element dynamic models, *Int. J. Numer. Methods Eng.* 102 (5) (2015) 1077–1110.
- [43] Todd Chapman, Philip Avery, Pat Collins, Charbel Farhat, Accelerated mesh sampling for the hyper reduction of nonlinear computational models, *Int. J. Numer. Methods Eng.* 109 (12) (2017) 1623–1654.